PART FOUR · 第四部分

Hooks: Claude's Deterministic Enforcement System · Hooks: Claude 的确定性执行体系

> The probabilistic model needs deterministic edges. Hooks are how the edges get built.

概率性模型需要确定性边缘。Hooks,就是构筑这些边缘的途径。

CHAPTER FIFTEEN · 第十五章

Hooks Fundamentals: Deterministic Behavior in a Probabilistic System · Hooks 基础: 概率性系统中的确定性行为

> *"Trust, but verify. The verification is what makes the trust possible."*
>
> — attributed to Ronald Reagan, who borrowed it from a Russian proverb

"信任,但要核实。核实,才让信任成为可能。"

—— 通常归于罗纳德·里根(他借自一则俄罗斯谚语)

← 前一页 后一页 →

> Skills and subagents, the two pillars covered so far, share a fundamental property. Both operate inside the probabilistic space of the language model. Both shape what Claude does, but neither imposes a hard constraint on what Claude must or must not do. A skill, however well written, instructs Claude through prose; the model interprets the prose and behaves accordingly, but the model retains the freedom to interpret. A subagent runs in its own context, but it is still a language model, and its behavior is still shaped by probability rather than by mechanical rule.

此前已经讲过的两根支柱——skill 与 subagent——共享一个根本属性:两者都跑在语言模型的"概率性空间"内部。两者都在塑造 Claude 的行为,但都没有对"Claude 必须做、不得做"施加硬约束。一条 skill 写得再好,也只是用散文指引 Claude;模型在解读这些散文之后再按解读行事,而解读的自由仍归模型所有。一个 subagent 跑在它自己的上下文里,但它仍是一个语言模型,其行为仍由概率塑造,而非由机械规则塑造。

> For most of the work a developer asks Claude Code to perform, this probabilistic shaping is exactly right. The flexibility of the model is what allows it to handle the open-ended variety of real requests. But there is a class of requirements where probabilistic shaping is insufficient. The class is small but important, and the requirements in it share a common shape: the failure rate must be zero. Tests must run before commits. Secrets must never reach disk. Production configuration must never be edited without a human in the loop. Linters must run after every code edit. For these requirements, no amount of careful prompting is good enough, because careful prompting can fail two percent of the time and the requirement does not tolerate two percent failure.

对开发者请 Claude Code 做的大多数工作而言,这种"概率性塑造"恰到好处。模型的灵活性,正是它能应对真实请求之"开放性、多样性"的原因。但仍有一类需求,是概率性塑造兜不住的。这条类很小,却至关重要——其中的需求都共享同一种形态:失效概率必须为 0。提交前必须先跑测试;秘密数据绝不许落盘;生产配置绝不能在没有"人在回路"的情况下被改;每次代码编辑后都必须跑 linter。对于这一类需求,再小心的 prompt 也不够用,因为再小心的 prompt 也会有 2% 的概率失败,而这类需求不容 2% 的失败。

> This chapter introduces the third pillar of the framework, which exists precisely to handle this class of requirement. Hooks are user-defined shell commands that Claude Code executes automatically at specific moments in its lifecycle. They run outside the language model. They cannot be talked out of running by clever prompts. They cannot be forgotten by Claude. They run, and their result either allows the lifecycle to continue or interrupts it. The discipline of using hooks well is the discipline of identifying which requirements need this kind of guarantee and then encoding the guarantee in shell commands that Claude Code will honor whether the model wants to or not.

本章要介绍的,是框架的第三根支柱——它存在的全部理由,就是去应对这一类需求。Hooks,就是由用户定义的 shell 命令,由 Claude Code 在其生命周期的特定时刻自动执行。它们跑在语言模型之外,无法被"巧妙的 prompt"说服不去跑,也不会被 Claude"忘记"。它们会跑——而其结果,要么让生命周期继续,要么把它打断。把 hook 用好这门功夫,关键在于两件事:一是识别"哪些需求需要这种保证",二是把这种保证编码成 shell 命令,让 Claude Code 无论模型想不想,都得照办。

> Begin with the mechanics. A hook is configured in a settings file. The most common location is the project's settings file, which lives at `.claude/settings.json` in the project root. User-level hooks, which apply to every project, live at `~/.claude/settings.json`. The settings file is JSON, and the hooks live under a top-level `hooks` key. Each hook entry specifies an event, an optional matcher that filters which tool calls the hook applies to, and a

从机制说起。Hook 在一份 settings 文件里配置。最常见的位置,是项目的 settings 文件——它位于项目根目录的 `.claude/settings.json`。用户级 hook 适用于所有项目,位于 `~/.claude/settings.json`。Settings 文件是 JSON 格式,所有 hook 都在顶层 `hooks` 键之下。每条 hook 项声明一个事件、一个可选的 matcher(用于过滤"哪些工具调用"会触发本 hook),以及一个

command to run when the event fires. The simplest possible hook configuration looks like this.

在事件触发时运行的命令。最简单的 hook 配置,看起来是这样的。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"$CLAUDE_TOOL_INPUT_FILE_PATH\""
          }
        ]
      }
    ]
  }
}
```

(配置含义:在 PostToolUse 事件上挂一个 hook,matcher 限定只在 Edit 和 Write 工具上触发,实际命令调用 Prettier 对被编辑/写入的文件就地格式化。)

Reading the Configuration · 解读配置

Read this configuration carefully, because the structure is identical for every hook the rest of this part of the book will introduce. The top-level `hooks` key holds an object whose keys are event names. The event name in this case is `PostToolUse`, which fires after a tool call has completed successfully. Under that event name is an array of hook configurations. Each configuration has an optional matcher, which uses regular expression syntax to filter which tools this hook should apply to; here, `Edit` and `Write` are the only matched tools, so the hook fires after edits and writes but not after, say, `Bash` or `Read` calls. The `hooks` array under that contains the actual commands to execute. Each command has a `type`, almost always `command` for shell execution, and the `command` itself, which is run with the standard shell.

请把这份配置仔细读一遍——这一部分后续介绍的每一条 hook,都共用同一份结构。顶层 `hooks` 键持有的是一个对象,对象的键是事件名。这里的事件名是 `PostToolUse`——它在一次工具调用成功完成之后触发。该事件名之下,是一个 hook 配置数组。每条配置含一个可选的 matcher,使用正则语法过滤"哪些工具会触发本 hook";这里只匹配 `Edit` 和 `Write`,所以 hook 只在"编辑/写入"之后跑,而不会在 `Bash`、`Read` 之类调用之后跑。它下面的 `hooks` 数组,才是真正要执行的命令。每条命令有一个 `type`(对 shell 执行而言几乎总是 `command`),以及具体的 `command` 字符串本身——它会用标准 shell 去执行。

The hook configuration above is small but functional. Its effect: every time Claude Code edits or writes a file, the file is automatically passed through Prettier, which formats it according to whatever Prettier configuration the project has. The developer never has to remember to run Prettier. Claude never has to remember either. The hook remembers, because the hook is not part of the language model's loop. The hook is

上面这条 hook 配置虽小,却已能"做事"。它的效果是:每逢 Claude Code 编辑或写入一个文件,该文件都会被自动交由 Prettier 处理,按项目自带的 Prettier 配置就地格式化。开发者不必记得去跑 Prettier;Claude 也不必记得。Hook 替大家记着——因为它不属于语言模型那条 loop。Hook 是

← 前一页

后一页 →

> wrapped around the loop, at a specific point in the lifecycle, and it runs every time the lifecycle reaches that point, no matter what.

"裹"在 loop 之外、挂在生命周期特定节点上的一道关卡——只要生命周期走到那一站,它就会跑,无论发生什么。

The Lifecycle · 生命周期

> The lifecycle is the grid of available moments where hooks can attach. Every Claude Code session passes through a sequence of named events, and a hook can be bound to any of them. The most common events, and the ones that almost every working developer eventually uses, are these. `PreToolUse` fires before a tool is about to be invoked, with the tool's name and input available to the hook. `PostToolUse` fires after a tool has completed successfully. `UserPromptSubmit` fires when the user submits a prompt, before Claude processes it. `Stop` fires when Claude finishes generating a response. `SubagentStop` fires when a subagent completes its work, before its result returns to the main session. `SessionStart` fires when a new session begins or is restored. `SessionEnd` fires when a session terminates. `Notification` fires when Claude needs to alert the user about something, such as needing approval. `PreCompact` fires just before Claude compacts its context to save space.

生命周期,就是 hook 可以挂上去的那张"时刻表"。每个 Claude Code session 都会经过一连串具名事件,hook 可以绑定到其中任何一个。最常用、也是几乎所有在岗开发者迟早会用上的那些事件,大致是这些:`PreToolUse`,在工具即将被调用前触发,工具名与输入对 hook 可见;`PostToolUse`,在一次工具调用成功完成后触发;`UserPromptSubmit`,在用户提交 prompt、Claude 尚未处理时触发;`Stop`,在 Claude 生成完一次响应时触发;`SubagentStop`,在 subagent 完成其工作、结果尚未回传主会话前触发;`SessionStart`,在一次新会话开始或被恢复时触发;`SessionEnd`,在会话终止时触发;`Notification`,在 Claude 需要向用户告警(例如请求审批)时触发;`PreCompact`,在 Claude 为腾出空间而压缩上下文之前触发。

> Each event has a slightly different shape of input passed to the hook on standard input as JSON, and a slightly different shape of expected output. `PreToolUse` hooks receive the tool name, the tool input, and the session context, and can decide to allow, deny, ask the user about the action, or defer the decision back to the agent loop. `PostToolUse` hooks receive the tool name, the input, and the response, and can issue feedback to Claude or block subsequent steps. `UserPromptSubmit` hooks receive the prompt text and can either block it or inject additional context for Claude to read. `SessionStart` hooks receive minimal context but can produce output that gets injected into the session's starting context, which is useful for loading project-specific reminders or recent git information at every session start.

每个事件,通过标准输入喂给 hook 的 JSON"输入形状"略有不同,所期待的"输出形状"也略有不同。`PreToolUse` hook 拿到工具名、工具输入、会话上下文,并据此决定:放行、拒绝、向用户确认,或把决定再交回 agent loop。`PostToolUse` hook 拿到工具名、输入、响应,可以向 Claude 发出反馈,或拦截后续步骤。`UserPromptSubmit` hook 拿到 prompt 文本,既可以拦截它,也可以向其中注入额外的上下文供 Claude 读取。`SessionStart` hook 拿到的上下文很少,但它可以产出一段输出,这段输出会被注入会话的"启动上下文"中——这件事在每个 session 一开始时,用来加载项目级提示或最近的 git 信息,非常顺手。

> The mechanism by which a hook signals its decision is straightforward. The hook is a shell command. Its standard output and standard error are captured. Its exit code is checked. Exit code zero means success, and the lifecycle continues without modification. Exit code two has a special meaning: in `PreToolUse` hooks the action is blocked; in `PostToolUse` hooks the action has already happened, so exit code two cannot block it but instead surfaces the standard error output back to Claude as feedback, and whatever the hook printed to standard error is surfaced back to Claude as feedback about why the block happened. Other non-zero exit codes are treated as

Hook 表达"它做了什么决定"的机制,非常直白。Hook 就是一条 shell 命令;它的标准输出与标准错误会被捕获,它的退出码会被检查。退出码 0 代表"成功",生命周期照常推进、不做修改。退出码 2 有特殊含义:对 `PreToolUse` 而言,这次动作被拦截;对 `PostToolUse` 而言,这次动作已经发生,退出码 2 拦不住了——它转而把标准错误里写的东西当作"反馈"回送给 Claude,告诉它"为什么被拦"。其他非零退出码,会被视作

← 前一页

后一页 →

> errors but do not block; they are logged. For richer control, the hook can also print structured JSON to standard output, which lets the hook approve or deny the action explicitly, modify the tool input before it runs, or inject additional context. The simple exit-code path covers most needs. The JSON path is for when the hook needs to do something more sophisticated than a binary allow or block.

"错误",只记录日志、并不拦截。要做更精细的控制,hook 还可以向标准输出打印一段结构化 JSON——这样它就能显式地"批准/拒绝"这次动作、在工具运行前改写其输入,或注入额外上下文。简单的"退出码路径"足以覆盖多数需求;JSON 路径,留给那些需要"比二元放行/拦截再复杂一点"的动作去用。

Exit Codes in Action · 退出码:实战

> A worked example using exit codes will make the blocking pattern concrete. Suppose the developer wants to ensure that Claude Code never edits the `.env` file in any project. A small hook accomplishes this. The hook is configured under `PreToolUse`, with a matcher of `Edit` or `Write`, and the command is a small shell expression that reads the tool input on standard input, checks whether the file path is `.env`, and exits with code two if so.

用退出码做一个完整例子,让"拦截"模式具象化。假设开发者希望"在所有项目里,Claude Code 都不许编辑 `.env`"。一条很小的 hook 就能办到。它配在 `PreToolUse` 下,matcher 写 `Edit|Write`,命令则是一段小巧的 shell 表达式——从标准输入读工具输入,判断文件路径是不是 `.env`,是则退出码 2。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -e '.tool_input.file_path | test(\"\\\\.env$\")' >/dev/null && { echo 'Editing .env files
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 PreToolUse 上挂一个 hook,matcher 限定 Edit 和 Write 工具;命令通过 `jq` 解析工具输入,若文件路径以 `.env` 结尾,则往标准错误写一条拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

The shell expression in that command is dense, but its logic is simple. It reads the JSON input from standard input, uses `jq` to test whether the tool input's file path matches the regular expression for `.env` files, and if it does, prints an error message to standard error and exits with code two, which blocks the operation. If the file is not a `.env` file, the hook exits with code zero, allowing the operation. The model receives the standard error message as feedback and can explain to the user that the `.env` file is protected. The user gets a deterministic guarantee: no matter what Claude wants to do, no

上面命令里那段 shell 表达式,乍看很紧,但其逻辑很直白:从标准输入读入 JSON,用 `jq` 测一下"工具输入里的文件路径"是否匹配 `.env` 文件的正则;若匹配,则向标准错误打一条错误消息,再以退出码 2 退出——动作被拦;若不是 `.env` 文件,则以退出码 0 退出,放行。模型会把那段标准错误消息当作"反馈"收下,并向用户解释:这个 `.env` 文件是受保护的。用户得到的,是一条确定性的保证:无论 Claude 想做什么,

← 前一页 后一页 →

matter how the user phrased the request, no matter what the system prompt or skills say, the `.env` file cannot be modified.

`.env` 文件都改不了——无论用户措辞如何,无论 system prompt 或 skill 怎么说,都改不了。

That guarantee is the whole point. The probabilistic model is left to its probabilistic work, and the deterministic requirement is handled deterministically, in its proper place. The model is freed from having to be vigilant about something it does not need to be vigilant about, because the hook is vigilant on its behalf. The developer is freed from having to verify on every interaction, because the verification is mechanical.

这才是这份保证的全部意义。概率性的模型,继续做它概率性的工作;而"必须确定性"的那件事,被放在它该在的地方,确定性地被处理掉。模型不必再为"它本不必警惕的事"而保持警惕——因为 hook 替它警惕。开发者也免于"每次交互都要去验证"——验证已被机械化了。

A second mechanism worth knowing about is the JSON output format, which the hook can use for richer control than a binary allow or block. When a hook writes a JSON object to standard output and exits with code zero, Claude Code parses the JSON for instructions about how to handle the action. The most useful field for `PreToolUse` hooks is `hookSpecificOutput`, which carries a `permissionDecision` of `allow`, `deny`, or `ask`, plus an optional `updatedInput` field that lets the hook modify the tool input before it runs. This last capability is striking on first encounter. A hook can intercept a tool call, decide that the call is acceptable but the input should be modified, and rewrite the input on the fly. The model never sees the modification. The tool runs with the rewritten input. A hook that wraps every `Bash` call to ensure that `npm install` runs with the `--no-save` flag would use this mechanism, intercepting the call, rewriting the command to add the flag, and letting the modified command proceed. The capability is powerful, and it should be used sparingly, because invisible modifications to tool input can produce confusing sessions if the developer forgets the hook is there. But when the use case is right, the JSON output format is the cleanest way to express it.

还有第二条机制值得知道——JSON 输出格式,让 hook 能做的控制远比"二元放行/拦截"更精细。当一条 hook 向标准输出写一个 JSON 对象,并以退出码 0 退出时,Claude Code 会解析这份 JSON,把它当作"如何处理这次动作"的指令。对 `PreToolUse` hook 而言,最有用的字段是 `hookSpecificOutput`,它带一个 `permissionDecision`(值是 `allow`、`deny`、`ask` 之一),并带一个可选的 `updatedInput` 字段,允许 hook 在工具实际运行前改写其输入。最后这项能力,初见会令人眼前一亮:hook 可以拦截一次工具调用,认定"调用本身没问题,但输入应当被改",并即时改写输入。模型完全看不到这次改写;工具以被改写后的输入跑。比如一条 hook 想"包住每一次 `Bash` 调用,确保 `npm install` 都带上 `--no-save`"——它就会用这条机制:拦截那次调用,改写命令加上这个 flag,然后让改写

后的命令放行。这项能力很强,但要节制使用——"对工具输入做不可见的改写"很容易让 session 变得费解,只要开发者忘了 hook 还在那儿。因此,只有在用例对的时候,JSON 输出格式才是最干净的选择。

The Settings Hierarchy · Settings 层级

> A short word on the settings hierarchy. Hooks can be configured at three levels: enterprise policy, user-level, and project-level. Enterprise policy settings are managed by an organization's administrators and apply to every project a developer opens. User-level settings live in `~/.claude/settings.json` and apply to every project the user opens. Project-level settings live in `.claude/settings.json` at the root of a specific project and apply only inside that project. There is also a project-local file at `.claude/settings.local.json` (which the project's `.gitignore` should exclude), intended for personal hooks that the developer wants in this project but does not want to commit to the

简短聊一下 settings 的层级。Hook 可以在三个层级上配置:企业策略、用户级、项目级。企业策略的 settings 由组织的管理员掌管,适用于该开发者打开的每一个项目。用户级 settings 位于 `~/.claude/settings.json`,对用户打开的每一个项目都生效。项目级 settings 位于具体项目根目录的 `.claude/settings.json`,只对该项目内部生效。还有一份项目本地文件 `.claude/settings.local.json`(项目的 `.gitignore` 应当排除它)——它专门放"想在本项目用、但不想提交到

← 前一页

后一页 →

> repository. The hierarchy is additive: hooks from all applicable levels run on every event, in a defined order, and any one of them can block. The developer should think carefully about which level a given hook belongs at. A security hook that protects against `rm -rf` belongs at the user level, because it should apply everywhere. A code-formatting hook that runs Prettier with project-specific configuration belongs at the project level, because the configuration is project-specific. A personal hook that sends desktop notifications when Claude needs input belongs in `settings.local.json`, because it is the developer's personal preference and not something her teammates need or want.

仓库里去"的个人 hook。这个层级是叠加式的:每个事件触发时,所有适用层级的 hook 都会按一个既定的顺序跑,而其中任何一条都可以拦截。开发者应该仔细想一想:某条 hook 应该住在哪一层?防 `rm -rf` 的安全 hook,应当住在用户层——它该到处生效;按项目级 Prettier 配置跑格式化的 hook,应当住在项目层,因为配置是项目特定的;在 Claude 需要输入时弹桌面通知的个人 hook,应当住在 `settings.local.json`——它纯粹是这位开发者的个人偏好,队友既不需要也不希望它出现。

> Understanding where each hook belongs is more than an organizational nicety. It directly affects how the hook is shared, how it is reviewed, and how easily it can be adjusted. A hook in the wrong place produces friction long after it was installed. A user-level hook that should have been project-level forces every other project to deal with conventions that only one project actually wants. A project-level hook that should have been user-level fails to protect the developer when she switches to a different project. A personal hook in the wrong place gets committed to the repository where her teammates encounter it as an unexplained presence. None of these mistakes is catastrophic, but each of them produces low-grade ongoing annoyance, and the annoyance can be avoided entirely by spending thirty seconds on the placement decision when the hook is being installed.

"hook 该住哪一层"这件事,理解到位的好处,远超"组织上好看"那么简单。它会直接决定 hook 如何被共享、如何被评审、调整起来有多容易。住错层级的 hook,会在安装之后很久还在制造摩擦。本该是项目级的 hook 放在用户层,会让其他所有项目都被"这一个项目才需要的规约"绑架;本该是用户层的 hook 放在项目层,会让开发者在切换到别的项目时失去那条保护;个人 hook 放错地方,会被提交进仓库,队友打开代码时一脸问号地撞上这条来历不明的存在。这些错误,没有哪个是灾难性的,但每一条都会带来低烈度、持续性的烦人;在 hook 安装的当下,花三十秒想清楚"该放哪",就能把这些烦人彻底省下。

> The relationship between hooks and the agent loop deserves explicit attention, because it is the relationship that makes the entire architecture work. The agent loop, recall from Chapter Two, is the cycle of think, act, observe that the language model runs through every turn. Every tool call happens inside the act phase. Every observation of a tool result happens inside the observe phase. Hooks attach to the boundaries of these phases. `PreToolUse` hooks fire just before the act phase invokes a tool. `PostToolUse` hooks fire just after the act phase has completed and the result is being returned to the observe phase. The hooks are, in this sense, the boundary between the model's probabilistic decisions and the deterministic environment that surrounds them. They are not part of the model's loop. They are part of the framework that the loop runs inside, and the framework can impose constraints that the model has no way to evade.

Hook 与 agent loop 之间的关系,值得专门提一笔——因为这层关系,正是整副架构"能跑起来"的根本。回顾第二章:agent loop,就是语言模型每轮跑一次的"思考—行动—观察"循环。每次工具调用发生在"行动"阶段;每次对工具结果的观察,发生在"观察"阶段。Hook 挂在这些阶段的边界上——`PreToolUse` 钩在"行动"阶段即将调用工具之前;`PostToolUse` 钩在"行动"阶段完成、结果正被送回"观察"阶段之际。Hook 在这个意义上,就是"模型的概率性决策"与"环绕模型的确定性环境"之间的那道边界。它不属于模型的 loop;它属于"loop 跑在其中的"那副框架。框架可以施加模型无论如何也绕不开的约束。

← 前一页

后一页 →

> A small clarification about the interactive `/hooks` command is worth including, because new users sometimes overlook it and pay an unnecessary cost. The `/hooks` command, run inside Claude Code, opens an interactive editor for the hook configuration. The editor lists every event, shows the hooks currently bound to each event, and offers menu options to add, modify, or remove hooks without having to edit the JSON configuration file directly. For developers who are new to hooks and not yet comfortable hand-editing JSON, the interactive editor is the friendlier path. For developers who are comfortable with JSON, direct editing of the settings file is faster, especially for non-trivial hooks. Both paths produce identical results, and the developer can switch between them depending on what is easier for the specific change she is making. The interactive editor is also useful as a verification tool: running `/hooks` in a session is a quick way to confirm that a hook configuration is loaded and active, which can save time when debugging why a hook seems not to be firing. Two related slash commands are worth knowing about as well. The `/agents` command lists the subagents available in the current session and lets the developer inspect their definitions interactively. The `/doctor` command reports on the current installation and surfaces the most common configuration problems before they become a debugging session. A common pitfall worth flagging here is forgetting that newly added hooks require a session restart before they take effect; configuration changes do not retroactively apply to a running session, and the developer who edits her hooks and then expects the changes to be live should restart Claude Code or open a new session before testing.

有一条关于交互式 `/hooks` 命令的小说明值得补上——新用户有时会忽略它,因而多付不必要的代价。在 Claude Code 里运行 `/hooks` 命令,会打开一个针对 hook 配置的交互式编辑器。编辑器会列出所有事件、显示当前绑定到每个事件上的 hook,并提供菜单选项,让你不必直接编辑那份 JSON 配置文件,就能增、改、删 hook。对刚接触 hook、还没那么习惯手写 JSON 的开发者来说,这条交互式路径更友好;对写 JSON 已经顺手的人来说,直接编辑 settings 文件更快,尤其是当 hook 不那么简单的时候。两条路径产出完全一致,开发者可以按"这次具体改起来哪种更顺手"自由切换。交互式编辑器同时也是一条很好的验证工具——在 session 里跑一次 `/hooks`,是快速确认"hook 配置已加载并激活"的最快方式,在排查"hook 似乎没触发"时能省不少时间。还有两条相关的斜杠命令也值得知道:`/agents` 命令会列出当前 session 中可用的 subagent,并让你交互式地检视它们的定义;`/doctor` 命令则会汇报当前安装的状态,把最常见的配置问题摆在台面上,免得它们拖到要专门 debug 的地步。此处还有一条常见的坑要专门提一句:刚加进去的 hook,需要重启 session 才会生效;配置变更不会追溯性地作用到正在运行的 session 上。因此,改完 hook 又期望改动立即生效的开发者,应当先重启 Claude Code 或开一个新 session,再去测试。

A worked debugging walkthrough is worth including, because hooks can fail in subtle ways and the developer who has a debugging procedure in working memory will save herself hours when something goes wrong. The walkthrough below is from a real session. The developer had configured a `PostToolUse` hook to run Prettier after every file write. She tested it, it worked, and she committed the configuration. Two days later, she noticed that Prettier had stopped running. The files Claude was writing were no longer being formatted, even though the hook configuration appeared unchanged.

一个完整的 debug 走查,值得放在这里——hook 失败的形态往往很微妙,工作记忆里有一份 debug 流程的开发者,真出问题时会为自己省下好几个小时。下面这份走查,出自一次真实的 session。开发者配了一条 `PostToolUse` hook,让 Prettier 在每次写文件之后跑。测试过、能跑、提交了配置。两天后,她发现 Prettier 不再跑了:Claude 写出的文件不再被格式化,而 hook 配置看上去又没动过。

A Debugging Walkthrough · Debug 走查

← 前一页

后一页 →

She approached the problem methodically. The first step was to verify the hook was loaded. She ran the `/hooks` command inside Claude Code, which lists all configured hooks for the current session. The Prettier hook appeared in the list, which meant the configuration was being read. The second step was to verify the hook was running. She added a small log statement to the hook itself, writing a timestamp to a file every time the hook fired. She made an edit, and the log file gained a new entry. The hook was firing. The third step was to verify the hook was succeeding. She checked the file the hook had been formatting and saw that the formatting had not been applied. The hook was firing but not completing its work.

她按部就班地处理。第一步:确认 hook 已加载。她在 Claude Code 里跑了 `/hooks` 命令,这个命令会列出当前 session 中所有已配置的 hook。Prettier hook 出现在列表里——说明配置确实被读到了。第二步:确认 hook 在跑。她在 hook 本身里加了一行小日志,让 hook 每次触发时都向一个文件里写一行时间戳。她做了一次编辑,日志文件多了一行——说明 hook 在触发。第三步:确认 hook 在"成功"地完成。她去看 hook 应当去格式化的那个文件,发现格式没被应用——hook 在触发,但工作没做出来。

The diagnosis came on the fourth step. She invoked the hook directly from her terminal, passing in a JSON input that mimicked what Claude Code would send. The hook ran, attempted to invoke Prettier, and failed silently. The reason was that `npx prettier` required the project's `node_modules` to be installed, and a recent dependency cleanup had removed Prettier from the project's dependencies. The hook itself was correct. The environment it ran in had drifted out from under it. She reinstalled Prettier, ran her test again, and the formatting returned.

诊断落在第四步:她直接在自己的终端里调用 hook,给了一段模仿 Claude Code 实际发送的 JSON 输入。Hook 跑起来了,试图去调 Prettier,然后悄悄失败。原因在于,`npx prettier` 依赖项目里的 `node_modules` 已安装;而一次最近的依赖清理,把 Prettier 从项目的依赖里移走了。Hook 本身没错,是它所运行的那个"环境"在它脚下漂移了。她重装了 Prettier,再跑一遍测试,格式化回来了。

The lesson the developer took away was that hooks fail silently more often than they fail loudly, and the procedure for debugging them is to walk down the chain: configuration loaded, hook firing, hook executing, hook succeeding. Each step has a verification. Each verification narrows the problem space. The whole procedure takes ten minutes and resolves the large majority of hook problems, because the failure is almost always at one of the four checkpoints, and the checkpoints are easy to test in isolation.

开发者从中总结出的经验是:hook 失败时,十有八九是"静悄悄地失败",而非"响亮地失败";debug 它们的标准流程,就是沿这条链一路往下走——配置已加载?hook 在触发?hook 在执行?hook 在成功?每一站都有验证;每一处验证,都在缩窄问题空间。整个流程十分钟就能走

完,且能解决绝大多数 hook 问题——因为失败几乎总落在四道关卡的某一道上,而每道关卡都很容易单独去测。

> A reflection before this chapter closes, on where hooks fit in the broader architecture of the book. Skills shape what Claude knows. Subagents shape who, in effect, is doing the work. Hooks shape what is permitted. The three together cover capability, specialization, and boundary. The fully engineered Claude Code environment uses all three, with hooks providing the deterministic edges that make it possible to trust the probabilistic systems in the middle. A developer who has installed skills and subagents but no hooks has a powerful environment that still depends on the model's judgment for safety-critical concerns, which is a reasonable trade-off but a real one. A developer who adds even three or four well-chosen hooks transforms her environment from a powerful assistant into a

在收束本章之前,先来点反思:hook 在本书更宏大的架构里,占的是哪一格?Skill 决定 Claude "知道什么";subagent 决定"实际上是谁在干活";hook 决定"什么被允许"。三者合起来,覆盖了"能力 / 专门化 / 边界"这三件事。一套完整工程化的 Claude Code 环境,会同时用上这三样:由 hook 提供确定性边缘,让中间那些概率性的系统变得可以信任。只装了 skill 和 subagent、没装 hook 的开发者,环境虽然强大,却仍要把"安全攸关的事"押在模型的判断上——这是一笔合理的交易,但也是一笔真实的赌注。再多加哪怕三四条精心挑选过的 hook,她的环境就会从"一个强大的助手"变成一个

← 前一页

后一页 →

> system she can rely on for things that genuinely matter, because the things that genuinely matter are now protected by mechanism rather than by hope.

"在真正重要的事上靠得住"的系统——因为真正重要的事,如今是由机制保护,而非由希望保护。

> The next chapter takes hook design from these mechanics into design patterns. The three archetypes that cover almost every useful hook are gates, guards, and notifiers, and the chapter ahead develops each pattern in depth, with worked examples ready to install. The reader who finishes the next chapter will have a small but capable set of hooks in her environment and the design vocabulary to add more whenever her work uncovers a new requirement that deserves deterministic handling.

下一章,要把 hook 的设计从"机制"层面带进"模式"层面。覆盖"几乎所有有用 hook"的三种原型,是 gate、guard、notifier;下一章会逐一深入展开,并配上"开箱即装"的具体例子。读完整章的读者,届时会在自己的环境里拿到一套虽小但能打的 hook,同时也会拥有一份"设计词汇"——任何时候,只要她的工作揭示出一个"值得被确定性对待"的新需求,她就能照着这套词汇再添新条。

← 前一页

Chapter 16 (Gates/Guards/Notifiers) →

CHAPTER SIXTEEN · 第十六章

Hook Design Patterns: Gates, Guards, and Notifiers · Hook 设计模式:门、哨、通

知器

> *"The shape of a tool is the shape of the problem it solves. Three problems, three shapes, no more."*
>
> — Henry Petroski, paraphrased

"工具的形状,就是它所解决的问题的形状。三个问题,三种形状,仅此而已。"

—— 亨利·彼得罗斯基(意译)

This chapter takes the hook mechanics of the previous chapter and develops them into three design patterns that, in combination, cover almost every hook a working developer will ever want to write. Naming the three patterns explicitly is valuable because it gives the developer a vocabulary for thinking about her hook design and a checklist for asking, when a new requirement arises, which of the three shapes the requirement most resembles. The three patterns are gates, guards, and notifiers, and each one solves a specific class of problem in a specific way. Once the patterns are in working memory, designing a new hook becomes a matter of recognizing which pattern fits and following the conventions for that pattern.

本章把上一章讲过的 hook 机制,发展为三种设计模式;三者在组合之下,几乎覆盖了一名在岗开发者要写的所有 hook。把这三种模式显式地命名,价值很大:它给了开发者一套"思考 hook 设计"的词汇,以及一份"清单"——每当出现一项新需求,都可以问:这条需求最像这三种形状里的哪一种?三种模式分别是 gate(门)、guard(哨)、notifier(通知器);每一种,都以自己的方式,解决某一类问题。一旦这三种模式进入工作记忆,设计新 hook 就变成"识别该用哪一格、按那格的约定写下去"的事了。

Gates · 门

Begin with gates. A gate is a hook that runs before an action and decides whether the action is allowed to proceed. The canonical lifecycle event for gates is `PreToolUse`, which fires before a tool call is dispatched. The gate inspects the tool input, applies whatever rule the developer cares about, and either lets the action through or blocks it. The blocking mechanism, as established in the previous chapter, is exit code two, with whatever explanation the developer wants surfaced to Claude on standard error.

先讲 gate。一条 gate 是一条"在动作发生前运行、决定该动作是否可以继续"的 hook。Gate 的标准生命周期事件是 `PreToolUse`——它会在一次工具调用被真正派出去之前触发。Gate 检查工具输入,套上开发者关心的那条规则,要么放行,要么拦截。拦截机制如上一章所约定:退出码 2,加上一段开发者希望送到 Claude 那里的、写在标准错误里的解释。

Gates are the most common and most consequential pattern in any hook library, because they are the pattern that turns wishes into rules. A developer who wishes that Claude would not commit directly to the main branch can encode the wish as a wish, by writing it into `CLAUDE.md` or into a skill, and Claude will follow the wish most of the time. The developer who wants the wish to become a rule writes a gate. The gate runs before any `git push` tool call, inspects the command for direct pushes to main, and exits with code two if the push targets main without an explicit override.

在任何 hook 库里,gate 都是最常用、也最重要的一格——因为它就是那条"把心愿变成规则"的模式。一个希望"Claude 不要直接 commit 到 main 分支"的开发者,可以把"心愿"当心愿写进 `CLAUDE.md` 或一条 skill 里,Claude 大多数时候会照办;一个希望"心愿成为规则"的开发者,则会写一条 gate。这条 gate 在每一次 `git push` 工具调用之前跑,检查命令里是不是直接 push 到 main,若是,又在没有显式 override 的情况下,就以退出码 2 退出。

> The wish is now a rule. Claude cannot push to main, full stop, regardless of what the user types or how the request is phrased.

于是"心愿"成了"规则"。Claude 不能 push 到 main——一字不差,一字不漏,无论用户敲下什么、措辞如何,都不行。

> A worked gate example will make the pattern concrete. The gate below blocks any `Bash` command that contains the dangerous `sudo rm -rf` pattern, because the developer has decided that this pattern should never be executed by Claude Code, no matter what.

用一个完整的 gate 例子把模式说透。下面这条 gate 会拦截任何包含 `sudo rm -rf` 这一危险模式的 `Bash` 命令——因为开发者已经决定:这个模式,无论什么情况,Claude Code 都不该执行。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' | grep -qE 'sudo[[:space:]]+rm[[:space:]]+-rf' && { echo 'Refusi
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 PreToolUse 上挂一条 gate,matcher 限定 Bash 工具;命令以 `jq` 从工具输入里抽取出命令字符串,用 `grep` 匹配 `sudo rm -rf` ;命中则往标准错误写拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

> The shell expression reads the tool input as JSON, extracts the command, checks for the dangerous pattern with `grep` , and if it matches, surfaces a clear refusal message and exits with code two. The block is total. The model cannot bypass it, because the gate is not a model concern. It is a system concern, enforced before the model's requested action ever reaches the shell.

这条 shell 表达式:以 JSON 形式读取工具输入,抽出命令,用 `grep` 检查是否含有危险模式;若命中,向标准错误写一条清晰的拒绝消息并以退出码 2 退出。拦截是绝对的。模型绕不开它——因为 gate 不是模型层面的事,而是系统层面的事:在模型请求的动作抵达 shell 之前,就已经被强制拦下。

The Discipline of a Good Gate · 一条好 gate 的纪律

> Gates have a discipline worth naming. A good gate has three properties. It is targeted; it fires only on the specific tool calls and inputs it is designed to evaluate, not on every Bash command in the universe. It is fast; it should add only a few milliseconds to any tool call, because hooks

> fire constantly and slowness in a hook becomes slowness in every interaction. It is informative; when it blocks an action, the message it surfaces explains why the block happened in language the developer or the model can understand

Gate 有几条值得点名的纪律。一条好的 gate,具备三条属性。其一,精准——它只在"自己设计来评估"的那一类工具调用与输入上触发,而非对全宇宙所有 Bash 命令都一视同仁;其二,快——它只能给任何一次工具调用增加若干毫秒的耗时,因为 hook 会反复触发,hook 一慢,所有交互都跟着慢;其三,清晰——当它拦截一次动作,送回的消息要把"为什么被拦"讲明白,所用的语言,开发者或模型都读得懂。

← 前一页 后一页 →

> and act on. A gate that blocks without explanation produces frustrating sessions. A gate that explains its reasoning turns a refused action into a learning moment.

让人能照做。一条 gate 拦下动作却没给解释,会让 session 越用越憋屈;而一条把理由讲清楚的 gate,会把"一次被拒"变成"一次学习"。

Guards · 哨

> Move to guards. A guard is a hook that runs after an action and enforces consistency. The canonical lifecycle event for guards is `PostToolUse`. The guard inspects the result of the action and either accepts it as-is, modifies the surrounding state, or signals back to Claude that the action's result was insufficient and more work is needed. The Prettier hook from the previous chapter is a classic guard: it runs after a file is written, formats the file according to project rules, and ensures that whatever Claude wrote ends up in the canonical project format whether Claude knew the format or not.

接下来说 guard。一条 guard,是一条"在动作发生后跑、用来强制一致性"的 hook。它对应的标准生命周期事件是 `PostToolUse`:guard 检查动作的产出,要么照单全收,要么改一改周边状态,要么向 Claude 回送"产出尚不充分,还得再补一轮"。上一章的 Prettier hook 就是 guard 的典型——它在一个文件被写完之后跑,按项目规约把文件格式化,确保 Claude 写出来的内容无论知不知道规约,最终都落到项目"正统"的格式上。

> Guards differ from gates in their relationship to the action. A gate is preventive; it stops things from happening. A guard is corrective; it lets things happen and then ensures the result fits the project's conventions. Guards are usually less dramatic than gates, because they do not block, but they are at least as valuable in practice. A library of well-chosen guards makes the project's output consistent in ways that no amount of careful prompting can match, because the guards run on every relevant action and apply their corrections without requiring anyone to remember.

Guard 与 gate 的差别,落在它们和"动作"的关系上:gate 是"预防性"的——它拦下事不让发生;guard 是"纠正性"的——它让事发生,再去保证结果贴合项目规约。Guard 看上去没有 gate 那么戏剧化——它不拦截——但实操中并不比 gate 逊色。一组精心挑选的 guard,会让项目的产出保持一致,这份一致,是再小心的 prompt 也换不来的——因为 guard 在每一个相关动作上都跑、它自我修正,不需要任何人"记得去做"。

> A worked guard example raises the Prettier pattern to something more complete. The guard below runs after any file edit or write, formats the file with Prettier, and additionally runs ESLint with the auto-fix flag to apply automatic linting fixes.

用一个完整的 guard 例子,把 Prettier 这条模式再推高一层。下面这条 guard 在任何文件编辑/写入之后跑:用 Prettier 格式化,再用带 `--fix` 标志的 ESLint 自动修一遍 lint。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\" 2>/dev/null || true"
          },
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); npx eslint --fix \"$FILE\" 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 PostToolUse 上挂两条 hook 形成的链,matcher 限定 Edit / Write / MultiEdit 工具;第一条把被编辑的文件交给 Prettier 格式化,第二条交给 ESLint 自动修——两者都吞掉错误并以退出码 0 退出,避免单次格式化失败打断 session。)

← 前一页

后一页 →

> Two hooks fire in sequence under the same matcher. The first runs Prettier. The second runs ESLint with auto-fix. Both extract the file path from the JSON input, both run their tool against the path, and both swallow errors and exit cleanly so that a transient formatter failure does not interrupt the session. The result: every file that Claude touches ends up formatted and lint-clean before the next prompt, regardless of what Claude wrote. The project's style is enforced mechanically, and the developer never has to think about formatting again.

两条 hook 在同一个 matcher 下依序触发:第一条跑 Prettier,第二条跑带 `--fix` 的 ESLint。两者都从 JSON 输入里抽出文件路径,都把工具对着那个路径跑一遍,又都吞掉错误、干净退出——这样一次偶发的格式化失败不会把 session 打断。效果是:Claude 碰过的每一个文件,在下一条 prompt 进来之前,都已经格式化、lint 干净——无论 Claude 一开始写成了什么样。项目的风格由此被机械化地强制,开发者再也不必把"格式化"这事再放到脑子里。

Notifiers in Detail · 通知器:详解

> Move to notifiers. A notifier is a hook that runs at a lifecycle event for the purpose of producing some output to a system other than Claude Code itself. The output might be a desktop notification, a log file entry, a Slack message, an audit record, or anything else that the developer wants the world outside the session to know about. Notifiers do not block. They do not modify behavior. They simply observe the lifecycle and react.

最后说 notifier。一条 notifier,是一条"在某个生命周期事件触发时,跑一遍、用来向 Claude Code 之外的某个系统产出一段输出"的 hook。这段输出,可能是一张桌面通知、一行日志、一条 Slack 消息、一份审计记录,也可以是任何"开发者希望 session 之外的世界知道"的事。Notifier 不拦截、不改行为,只是在生命周期里"观察一下、做出反应"。

> The most common notifier event is the `Notification` event itself, which fires when Claude wants to alert the user about something, often a permission request. The developer can attach a notifier to this event that produces a desktop pop-up, so that she does not have to be staring at the terminal to see when Claude needs her attention. `Stop` is another common notifier event; a notifier on `Stop` can log every completed task to an audit file or send a brief summary to a team channel. `SessionEnd` notifiers can save session statistics. `PostToolUse` notifiers can append every `Bash` command Claude ran to a permanent log, which is useful both for audit and for retrospective debugging.

最常见的 notifier 事件,就是 `Notification` 本身——它会在 Claude 想向用户告警(通常是请求授权)时触发。开发者可以在这个事件上挂一条 notifier,产出桌面弹窗,这样她就不必一直盯在终端上等 Claude 找她。`Stop` 也很常用——挂上去的 notifier 可以把每一项已完成的任务写进审计文件,或向团队频道发一条简短总结。`SessionEnd` notifier 可以把会话统计信息存档。`PostToolUse` notifier 可以把 Claude 跑过的每一条 `Bash` 命令追加到一份永久日志里——对审计、对事后回溯 debug,都有用。

> A worked notifier example combines two of these patterns. The configuration below sends a desktop notification when Claude needs the user's input, and logs every Bash command Claude runs to a persistent log file.

一个完整的 notifier 例子,把上面两种用法合到一起。下面这份配置:在 Claude 需要用户输入时弹一条桌面通知;同时把 Claude 跑过的每一条 Bash 命令,写进一份持久化的日志文件。

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.message' | xargs -I {} osascript -e 'display notification \"{}\" with title \"Claude
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' >> ~/.claude/command-log.txt"
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 Notification 事件上挂一条 notifier,从消息体里抽取出文本,交给 osascript 弹一条 macOS 桌面通知;在 PostToolUse 上挂另一条,matcher 限定 Bash 工具,把每次 Bash 命令追加写入 `~/.claude/command-log.txt` 。)

> The first hook fires whenever Claude needs the user's attention; the developer's desktop pops up a notification, and she can return to the terminal at her convenience instead of having to monitor it. The second hook fires after every Bash command and writes the session ID and the command to an audit log in the user's home directory. The log accumulates across sessions and projects, providing a complete record of every shell command Claude has executed under the developer's account.

第一条 hook 在 Claude 需要用户关注时触发——开发者的桌面会弹出一条通知,她可以在自己方便时再回到终端,不必一直盯着。第二条 hook 在每次 Bash 命令后触发,把"session ID + 命令"写进用户主目录下的一份审计日志。这份日志跨 session、跨项目累积,等于一份"这位开发者的账户下,Claude 执行过哪些 shell 命令"的完整账本。

> A discipline applies to notifiers that the developer should keep in mind. Notifiers should be silent in the success case. They produce their side effect, and they exit cleanly. They do not produce stdout or stderr that the model might mistake for relevant output. A noisy notifier pollutes the session and degrades the experience. A silent notifier is invisible to the workflow except in the side effect it produces, which is exactly what the developer wants.

对 notifier,有一条应当记在心里的纪律:成功路径上,notifier 应当保持沉默。它产生它的副作用,然后干净地退出。它不应当把 stdout 或 stderr 写出来——因为模型可能误以为那是相关的输出。一条吵闹的 notifier 会污染 session、劣化体验;一条安静的 notifier,在工作流里几乎不可见,只在它所产生的那个副作用上"显形"——这正是开发者想要的。

> A second worked notifier example, in a slightly different configuration style, makes the discipline visible. Below, a notifier on the `Notification` event uses the Linux `notify-send` tool to produce a desktop notification, and a notifier on `PostToolUse` appends a session-aware line to a long-running bash history log.

第二条完整的 notifier 例子,采用略不同的配置形态,把这套纪律摆在明面上。下面的 notifier,在 `Notification` 事件上,用 Linux 的 `notify-send` 工具弹桌面通知;在 `PostToolUse` 上,把一条带 session 标记的行追加进一份"长期累积"的 bash 历史日志。

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Awaiting your input'"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '\"\\(.session_id) \\(.tool_input.command)\"' >> ~/.claude/bash-history.log"
```

```
          }
        ]
      }
    ]
  }
}
```

(配置含义:Notification 事件上的 notifier 用 `notify-send` 弹一条"Claude Code: Awaiting your input"的桌面通知;PostToolUse 上以 Bash 为 matcher 的 notifier,把 `session_id` 与命令拼成一行追加到 `~/.claude/bash-history.log`。)

← 前一页 后一页 →

A second discipline applies to all three patterns and is worth naming explicitly: keep hooks fast. Every hook fires on the lifecycle event it is bound to, and every hook adds latency to every event. A few milliseconds is invisible. A few hundred milliseconds is felt. A few seconds is unbearable, especially for events that fire frequently like `PostToolUse`. The developer should profile her hooks at install time, ensure they complete quickly, and pay particular attention to hooks that invoke heavy tools like full test suites or large linters. Heavy work, when it must run, often belongs in an asynchronous hook that fires the work and returns immediately, leaving the work to complete in the background. Asynchronous hooks are supported through the `async` configuration flag, which the developer can use when the work is genuinely independent of the immediate flow.

还有一条纪律,适用于全部三种模式,值得显式点出来:让 hook 保持快速。每条 hook 都会在它绑定到的那个生命周期事件上触发,每条 hook 都会给那次事件增加延迟。几毫秒,无感;几百毫秒,有感;几秒,会令人难以忍受——尤其对那些高频触发的事件,比如 `PostToolUse`。开发者应当在安装时对自己的 hook 做一下 profile,确保它们完成得够快,并对那些会调重型工具(完整测试套件、大型 linter)的 hook 多加留意。确实要跑的重活,通常属于"异步 hook"——它把任务发起,然后立刻返回,把后续完成的部分放到后台跑。异步 hook 借由 `async` 这个配置开关来支持,当某项任务确实独立于眼前这条流时,就可以用。

The Three-Pattern Vocabulary · 三模式词汇表

The three patterns, gates and guards and notifiers, are the basic vocabulary of hook design. With them in working memory, the developer can look at almost any requirement and immediately recognize which pattern fits. A requirement that says *no, do not, never* is almost always a gate. A requirement that says *always, ensure, conform* is almost always a guard. A requirement that says *tell me, log, alert* is almost always a notifier. The classification is not a rigid taxonomy. It is a thinking tool. It accelerates the design of new hooks by giving the developer a starting shape for any requirement she encounters, and that starting shape is, in most cases, the right shape.

这三种模式——gate、guard、notifier——就是 hook 设计的基本词汇表。把它们装进工作记忆之后,开发者再看几乎任何一项需求,都能立刻认出该归哪一格。一条措辞像"不许、不要、绝不能"的需求,几乎总是 gate;一条措辞像"始终要、务必、必须合于"的需求,几乎总是 guard;一条措辞像"告诉我、记下来、提醒我"的需求,几乎总是 notifier。这种分类不是僵硬的分类学,而是一个思考工具:它为开发者遇到的任何需求,都给出一个"起步形状",从而加速新 hook 的设计;而这个起步形状,大多数时候就是对的形状。

A worked extension of the guard pattern deserves attention here, because guards have a sophistication that gates and notifiers lack and that a developer should understand before designing a serious hook library. A guard can do more than just modify the file system after a tool runs. A guard can also report back to Claude through the JSON output format, telling Claude that something about the result of the tool call was unsatisfactory and that more work is needed. A guard configured this way is not merely corrective; it is conversational. It speaks back to the model and influences what the model does next.

guard 模式还有一条值得专门讲的扩展:guard 拥有一种 gate 与 notifier 都不具备的"复杂能力",在搭建严肃 hook 库之前,应当先把它想清楚。Guard 并不止于"在工具跑完之后改一改文件系统";它还可以借 JSON 输出格式向 Claude 回话——告诉它"这次工具调用的结果在某一点上不令人满意,还需要再补点活"。如此配置的 guard,就不再只是"纠正性"的,而是"对话性"的:它向模型回话,并影响模型下一步要做什么。

> The example below shows this pattern in action. The hook runs after every `Edit` or `Write` tool call. It runs the project's test suite, and if any tests fail, it surfaces the failure back to Claude with a directive that the failing

下面这个例子把这种模式跑起来给你看:这条 hook 在每一次 `Edit` 或 `Write` 之后触发——它跑项目的测试套件,只要有测试失败,就把失败回送给 Claude,附带一条指令,让模型去把"那些失败的

← 前一页

后一页 →

> tests should be fixed before the session continues. The mechanism is the JSON decision-block format, which carries a `decision` of `block` and a `reason` that explains what needs to happen.

测试修好,再继续推进 session"。机制就是 JSON 决策–拦截格式:它带一个 `decision` 取值 `block`,以及一段 `reason` 说明该做什么。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "if ! npm test --silent >/tmp/test-out 2>&1; then echo '{\"decision\":\"block\",\"reason\":\"
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 PostToolUse 上挂一条"对话式 guard",matcher 限定 Edit / Write / MultiEdit;命令跑 `npm test` 并把输出重定向到 `/tmp/test-out`。若测试失败,则向 stdout 打印一份 JSON 决策块,告知模型"测试失败,请先读 /tmp/test-out 并修好,再继续"。)

> The hook runs the test suite after each edit. If the tests pass, nothing happens; the hook exits with code zero and no JSON output, and Claude proceeds normally. If the tests fail, the hook prints the JSON decision-block, which Claude Code reads as a directive to surface the reason back to the model. The next message Claude sees contains the block reason, and Claude responds by reading the test output and fixing the failing tests. The result is that the test suite functions as a continuous check on every edit Claude makes; broken tests are surfaced immediately and addressed

before the session moves on. The developer who has installed this hook never has to remember to run tests. The hook remembers, and Claude responds.

这条 hook 在每次编辑之后跑一次测试套件。若测试通过,什么都没发生——hook 以退出码 0 退出、不输出任何 JSON,Claude 照常推进。若测试失败,hook 打印那段 JSON 决策块,Claude Code 把它读成"把 reason 回送给模型"的指令。Claude 下一条看到的消息里,会包含这条 block reason;它会读测试输出,去把失败的测试修好。效果就是:测试套件成了 Claude 每次编辑的"持续校验器"——坏的测试被立即翻到台面上,赶在 session 推进之前被处理。装上这条 hook 的开发者,从此不必再记得"要跑测试"——hook 替她记着,Claude 顺势响应。

A discipline applies to test-running guards that is worth naming, because the obvious implementation is often too slow. Running the full test suite after every edit is acceptable for projects with fast tests, but it becomes painful for projects whose tests take more than a few seconds. The right adaptation is to run only the tests that exercise the changed code, using whichever test selection mechanism the developer's test runner supports. For Jest and Vitest, this might mean using the related-files flag to identify tests that import the changed file. For Pytest, this might mean using a test collection plugin. The general principle is that the hook should run the

对"跑测试的 guard",有一条值得点名的纪律——因为最直白的实现往往太慢。"每次编辑都跑全套测试"——对测试跑得快的项目可以接受,但只要测试多花几秒,体验就会变得痛苦。正确的调校是:只跑"覆盖了被改代码"的那部分测试,用开发者所用测试运行器自带的"测试挑选"机制。对 Jest 和 Vitest 而言,可以用 "related–files" 标志去筛出"import 了被改文件"的测试;对 Pytest 而言,可以用一个 test collection 插件。一般原则是:hook 该跑的是

← 前一页

后一页 →

smallest set of tests that gives confidence about the change, not the full suite, because the latency penalty of running the full suite quickly outweighs the benefit of having the check run automatically.

"能对这次改动给出信心"的最小测试集,而非全套——因为"每次都跑全套"带来的延迟代价,很快就压过"自动跑校验"带来的收益。

Asynchronous Hooks · 异步 Hook

A short word on asynchronous hooks. Some work the developer wants done after a tool call does not need to block the lifecycle. Logging an audit record. Sending a backup. Updating a dashboard. None of these need to delay the session's progression. For these cases, Claude Code supports asynchronous hooks, configured by adding an `async` flag to the hook definition. An asynchronous hook is fired and forgotten; the lifecycle proceeds while the hook runs in the background. The cost is that the hook's output cannot influence the lifecycle, because by the time the hook finishes, the lifecycle has moved on. This is exactly the right trade-off for the categories of work that asynchronous hooks are meant to handle, where the work matters but the timing does not. The configuration looks like this.

关于异步 hook,简短说几句。有些"工具调用之后要做的工作"并不需要阻塞生命周期:记一条审计记录、推一份备份、更新一块仪表盘——这些都不该拖慢 session 的推进。Claude Code 为此支持"异步 hook",在 hook 定义里加一个 `async` 标志来启用。异步 hook 是"发完就忘"的:它把任务在后台触发,生命周期继续推进。代价是,hook 的输出无法再影响生命周期——因为等它跑完,生命周期早已走远。这对异步 hook 适合处理的那类工作而言,恰好是合适的取舍:事情要做,但不必卡在当下。配置长这样。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node $CLAUDE_PROJECT_DIR/.claude/hooks/sync-to-backup.js",
            "async": true,
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

(配置含义:在 PostToolUse 上挂一条异步 hook,matcher 限定 Edit / Write;命令以 `async: true` 标记,启动备份脚本后立刻把控制权交回 session。`timeout: 30` 字段把工作时长封顶在 30 秒;超时的备份脚本会被中止。)

The hook fires after every edit, runs the backup script in the background, and returns control to the session immediately. The timeout field caps the work at thirty seconds; if the backup script has not finished by then, it is terminated. The session continues without waiting. The asynchronous flag is the right choice for any hook whose work is genuinely independent of the immediate flow, and the developer who learns to

这条 hook 在每次编辑后触发,后台跑备份脚本,立刻把控制权交回 session。`timeout` 字段把任务时长封顶在 30 秒——若备份脚本届时仍未完成,就被中止。Session 不会等它。"异步标志"适合一切"工作本身确实独立于眼前这条流"的 hook;学会把

← 前一页

后一页 →

recognize which hooks belong asynchronous and which need to be synchronous will build pipelines that are both safe and fast.

同步、哪些适合异步这件事想清楚的开发者,会搭出既安全又快的流水线。

A reflection before this chapter ends, on what makes the gate-guard-notifier vocabulary so useful in practice. The vocabulary is small enough to hold in working memory, and it covers the cases that come up in real work. A developer who has internalized the three patterns can sit down with a new requirement, ask herself which of the three fits, and have a starting design within a minute. She does not have to re-derive the design space from first principles every time she wants to add a hook. The pattern catalog accelerates the design process, and accelerated design is what allows the developer to add hooks quickly enough that they get added at all. A developer without the vocabulary tends to put off hook work, because each hook feels like a fresh design problem. A developer with the vocabulary tends to add hooks freely, because each hook is a small variation on a familiar pattern.

本章收束前,先来反思一下:为什么"gate / guard / notifier"这套词汇在实务中如此有用?词汇量小到能装进工作记忆,却又覆盖了真实工作中会冒出来的那些情形。一个已经把这三种模式内化了的开发者,面对一项新需求,只需要问自己"它像哪一格",一分钟之内就能给出一

个起步设计。每加一条 hook,都不必从"第一性原理"重新推演整个设计空间。这份"模式目录"加速了设计过程——而正是这份加速,让开发者来得及"在需要的时候就把 hook 加进去"。一个没有这套词汇的开发者,往往把 hook 工作一拖再拖——因为每条 hook 看起来都像一道新设计题;一个有了这套词汇的开发者,加 hook 就很轻——因为每条 hook 都只是某个熟悉模式的小变体。

> A second reflection concerns the way the three patterns interact with the lifecycle events. Gates are usually `PreToolUse` hooks, because the natural moment to evaluate whether an action should be allowed is just before the action runs. Guards are usually `PostToolUse` hooks, because the natural moment to enforce consistency is just after the action has produced something to enforce on. Notifiers can attach to almost any event, because their job is to observe and report rather than to evaluate or correct. The pattern-event correspondence is not strict, but it is strong, and a developer reaching for the wrong event for her pattern often finds that the hook is harder to write than it needs to be. A gate written as a `PostToolUse` hook can detect a violation but cannot prevent it. A guard written as a `PreToolUse` hook can decline an action but cannot enforce a correction on a result that does not yet exist. Recognizing the natural pattern-event pairing is part of the craft, and pairing them correctly produces hooks that fit cleanly into the lifecycle.

第二条反思,关于三种模式与生命周期事件之间的"贴合"。Gate 通常挂在 `PreToolUse` ——评估"一次动作是否该被允许"的天然时机,就是动作跑起来之前的那一刹那。Guard 通常挂在 `PostToolUse` ——强制一致性的天然时机,是动作刚刚产出"可以去强制"的对象之后。Notifier 可以挂到几乎任何事件上——它的职责只是"观察并报告",既不评估,也不纠正。模式与事件之间的对应,不是死规,但确实强;一个把模式挂错事件的开发者,常会发现自己的 hook 写起来比实际更别扭。把 gate 写成 `PostToolUse` ——它只能察觉违规,却不能阻止;把 guard 写成 `PreToolUse` ——它能拒绝一个动作,却不能在一个尚不存在的"结果"上强加修正。认出"模式–事件"的天然配对,本就是这门手艺的一部分;一旦配对正确,hook 就会干干净净地嵌入生命周期。

> A short word about combining patterns within a single hook deserves attention, because the question comes up often. A hook that does both validation and correction, for instance, can be tempting; the developer might want a hook that, after a file is written, both formats it and rejects it if formatting fails. The temptation should be resisted in most cases. A hook that does two jobs is a hook that is harder to debug, harder to reason about,

关于"把多种模式合到同一条 hook 里"这件事,值得专门说一句——因为这种问题会反复冒出来。比如,有人很想写一条 hook,在文件写入后,既做格式化,又在格式化失败时拦截。这种诱惑在多数情况下都该抵抗。一条 hook 干两件事,就意味着它更难以 debug、更难讲清逻辑,

← 前一页

后一页 →

> and harder to disable when one of its jobs becomes inappropriate. The cleaner approach is to write two separate hooks, each with a single responsibility, and to bind them to the same event in sequence. The two hooks compose into the combined behavior the developer wanted, but each one remains independently understandable, testable, and maintainable. This single-responsibility discipline at the level of individual hooks is one of the small habits that distinguishes well-designed pipelines from those that have grown into tangled balls of multi-purpose scripts.

也更难在某一件事变得不再合适时单独关掉。更干净的做法是:写两条独立的、各自只负一责的 hook,按顺序绑到同一事件上。两条 hook 组合起来,确实就是开发者想要的那种"复合行为";但每一条都各自"可理解、可测试、可维护"。在"单条 hook 层面"贯彻"单一职责"这条纪律,是把"设计良好的流水线"与"长成乱麻的多用脚本"区分开的小习惯之一。

The next chapter takes the gate pattern in particular and develops it further. The most consequential application of gates in any production-grade Claude Code environment is in the security layer, where hooks protect against dangerous commands, sensitive file access, secret exposure, and other risks that the probabilistic model cannot be trusted to catch reliably. Building a serious security-hook bundle is the next step in this part of the book, and it is the work of the chapter ahead.

下一章要把 gate 模式再单独拎出来,推得更深。在任何"生产可上"的 Claude Code 环境里,gate 最具分量的一项应用,都在安全层——hook 在那里挡掉危险命令、敏感文件的访问、秘密数据的泄露,以及其他那些"概率性模型无法稳定地兜住"的风险。搭一套严肃的"安全 hook 套件",是本书这一部分的下一站,也是下一章要做的事。

← 前一页

Chapter 17 (Security Hooks) →

CHAPTER SEVENTEEN · 第十七章
Security and Permission Hooks: Protecting Your System · 安全与权限 Hook:保护你的系统

> *"The walls do not have to be high, but they must be where they are needed. A wall in the wrong place is decoration."*
>
> — a fortifications engineer, anonymous

"墙不必高,但必须建在该建的地方。建错位置的墙,只是装饰。"

—— 某位堡垒工程师,匿名

Of all the work hooks can do, the most consequential is security. A skill that produces excellent code is valuable. A subagent that runs a thorough review is valuable. But a hook that prevents a destructive command from executing, that catches a secret being written to disk, that stops a force-push from reaching production, is valuable in a different and more serious way. Security hooks are the layer of the engineered Claude Code environment where the cost of a single failure can be substantially higher than the entire benefit of the surrounding configuration. A developer can lose a day of work to a single `rm -rf` that should not have run. The hooks in this chapter exist to ensure that the day is not lost.

在 hook 能做的所有事情里,后果最重的一件,是安全。一条 skill 能产出优秀的代码,有价值;一个 subagent 能跑一次彻底的评审,也有价值。但一条 hook 能拦下"一次本不该执行、却差点执行掉的破坏性命令",能"在秘密数据落盘前一把抓住",能"阻止一次 force-push 抵达生产"——它的价值,以另一种更严肃的方式存在。安全 hook 是工程化 Claude Code 环境里这么一层:这里一次失败的代价,可以大幅超过整个周边配置带来的全部收益。一次不该跑却跑了的 `rm -rf`,能让开发者丢掉一整天的工作。本章要讲的这些 hook,存在的全部意义,就是确保那一天的工作不被丢掉。

This chapter builds a security-hook bundle. By the end of it, the reader will have installed five hooks that, together, raise the floor of safety in her Claude Code environment substantially. The five hooks address: dangerous shell commands, protected file paths, secret detection in writes, force-push prevention, and a permission-confirmation gate for high-impact operations. Each hook follows the gate pattern from the previous chapter. Each one is a small, focused expression that does one thing and does it deterministically.

本章要搭出一套"安全 hook 套件"。到本章结尾,读者会装好五条 hook——它们合起来,会显著抬高她 Claude Code 环境的"安全底线"。这五条 hook 分别解决:危险的 shell 命令、受保护的文件路径、写入内容中的秘密数据、force-push 拦截,以及一项"对高影响操作做权限确认"的 gate。每条 hook 都沿用上一章讲过的 gate 模式;每一条都是一条小巧、聚焦、确定性做事的小表达式。

> Begin with dangerous shell commands. The pattern is to inspect every `Bash` command before execution, looking for patterns that the developer has decided should never run. The list of dangerous patterns varies by environment, but a defensible default catches: `rm -rf` on root or home paths, `sudo` combined with destructive commands, `dd` writing to block devices, `mkfs` in any form, `chmod` with wide-open permissions on system paths, and any command that pipes from `curl` or `wget` directly into `bash`. The hook below covers these.

从"危险的 shell 命令"讲起。模式是:在每一条 `Bash` 命令执行之前,过一遍,看它是否落入"开发者已经决定永远不该跑"的那一沓模式。危险模式清单因环境而异,但有一份"说得过去的默认清单"值得守住:`rm -rf` 涉及根或主目录路径、`sudo` 与破坏性命令的组合、`dd` 写到块设备、任何形态的 `mkfs`、`chmod` 把系统路径的权限设得很开、以及把 `curl` 或 `wget` 的输出直接管道给 `bash` 的任何命令。下面这条 hook 覆盖了这些情况。

← 前一页

后一页 →


```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
          }
        ]
      }
    ]
  }
}
```

> (配置含义:在 PreToolUse 上挂一条 hook,matcher 限定 Bash 工具;命令调用项目 `.claude/hooks` 目录下的 `block-dangerous.sh` 脚本——把核心逻辑放进一个真正的脚本里,而不是塞在行内 shell 表达式中。)

```
#!/usr/bin/env bash
# block-dangerous.sh — refuses dangerous shell commands.
CMD=$(jq -r '.tool_input.command')

# Patterns that are always blocked.
PATTERNS=(
  'rm[[:space:]]+-rf?[[:space:]]+(/|~|/home|/Users|/etc|/var)'
  'sudo[[:space:]]+rm'
  'dd[[:space:]]+if=.*[[:space:]]+of=/dev/'
  'mkfs\.'
  'chmod[[:space:]]+777'
  'curl[[:space:]]+.*\|[[:space:]]*(bash|sh)'
```

```
  'wget[[:space:]]+.*\|[[:space:]]*(bash|sh)'
  ':\(\)\{[[:space:]]*:\|:&\}'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$CMD" | grep -qE "$PAT"; then
    echo "Refusing dangerous command. Pattern matched: $PAT" >&2
    exit 2
  fi
done

exit 0
```

(脚本含义:从 JSON 输入中读出 Bash 命令,遍历一个"黑名单正则"数组,只要命中任一条,就向标准错误写一条"Refusing dangerous command. Pattern matched: ..."消息,并以退出码 2 退出;都没命中则以退出码 0 放行。其中末项 `:(){:&}` 是经典的"fork 炸弹"模式。)

← 前一页

后一页 →

The script reads the command from the JSON input, walks an array of forbidden patterns, and exits with code two on the first match. The patterns are not exhaustive; the developer may add to them as she encounters new things she wants blocked. The shape of the script is what matters: it is a list of patterns, easy to extend, with a clear refusal message that surfaces to Claude on every block.

这段脚本从 JSON 输入中读出命令,在一组"禁模式"数组上挨个走一遍,任一命中就以退出码 2 退出。模式并未穷尽——开发者可以随时把自己新认定的危险模式加进去。脚本真正值得学的,是它的形状:一份"模式清单",易于扩展;而每条命中,都对应一条清晰可见的拒绝消息,会在每次拦截时被送到 Claude 那里。

Protecting File Paths · 受保护的文件路径

Move to protected file paths. The pattern is similar but applies to `Edit` and `Write` rather than `Bash`. The hook inspects the file path being written to and refuses if the path matches a protected list. The protected list typically includes `.env` files, lockfiles, the `.git` directory, production configuration files, and any file the developer has decided should require explicit human review for any modification.

下一条,讲"受保护的文件路径"。模式相似,但挂在 `Edit` / `Write` 上,而不是 `Bash`。Hook 检查"正被写入的文件路径",若落入受保护清单,就直接拒绝。这份清单通常包含:`.env` 文件、lockfile、`.git` 目录、生产配置文件,以及任何开发者已经认定为"任何改动都需显式人工复核"的文件。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
          }
```

```
        ]
      }
    ]
  }
}
```

(配置含义:在 PreToolUse 上挂一条 hook,matcher 限定 Edit / Write / MultiEdit 工具;命令调用 `protect-files.sh` 脚本。)

```
#!/usr/bin/env bash
# protect-files.sh — refuses edits to protected file paths.
FILE=$(jq -r '.tool_input.file_path')

PATTERNS=(
  '\.env(\.|$)'
  '\.git/'
  'package-lock\.json$'
  'yarn\.lock$'
  'Cargo\.lock$'
  'production\.ya?ml$'
  '\.pem$'
  '\.key$'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$FILE" | grep -qE "$PAT"; then
    echo "Refusing to edit protected file: $FILE" >&2
    echo "Pattern matched: $PAT" >&2
    exit 2
  fi
done

exit 0
```

(脚本含义:从 JSON 输入里抽取出文件路径,遍历一组"路径正则",命中任一就向标准错误写明"被保护的文件 / 命中的模式"并以退出码 2 退出;否则以退出码 0 放行。)

← 前一页

后一页 →

> The protection is total. No matter what the user asks, no matter what skill or subagent is firing, the listed paths cannot be edited by Claude. If the developer genuinely needs to modify a protected file, she does so manually outside the hook, or temporarily disables the hook for a single session and re-enables it afterward. The friction is a feature; the protection is a feature; the explicit deliberateness of any modification to these files is a feature.

这种保护是绝对的——无论用户怎么说,无论哪条 skill 或哪个 subagent 在触发,清单里的路径,Claude 都改不了。如果开发者真的需要去改某个受保护的文件,她就在 hook 之外手动改,或者为了单个 session 临时关掉 hook,改完再开回来。这种"摩擦"是 feature,这种"保护"是 feature,对"修改这些文件"这件事的"刻意的审慎"也是 feature。

Secret Detection · 秘密数据检测

> Move to secret detection. This hook is harder to write well, because secrets come in many forms and false positives are annoying. A working compromise looks for the most obvious patterns: AWS keys, GitHub tokens, OpenAI keys, generic high-entropy strings that look like API keys, and the explicit string `private_key` or `BEGIN PRIVATE KEY`. When a write contains any of these patterns, the hook refuses.

下一条,讲"秘密数据检测"。这条 hook 比较难写好——因为秘密数据形态万千,而误报又很烦人。一个"能跑得起来"的折中,就是去匹配那些最显眼的模式:AWS key、GitHub token、OpenAI key、看起来像 API key 的高熵字符串,以及显式的 `private_key` 或 `BEGIN PRIVATE KEY`。一旦写入内容里出现这些模式,hook 就拒绝。

```
#!/usr/bin/env bash
# block-secrets.sh — refuses writes that appear to contain secrets.
CONTENT=$(jq -r '.tool_input.content // .tool_input.new_string // ""')

PATTERNS=(
  'AKIA[0-9A-Z]{16}'
  'ghp_[A-Za-z0-9]{36,}'
  'sk-[A-Za-z0-9]{32,}'
  'BEGIN[[:space:]]+(RSA|EC|DSA|OPENSSH|PRIVATE)[[:space:]]+KEY'
  'aws_secret_access_key[[:space:]]*=[[:space:]]*[A-Za-z0-9+/]{40}'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$CONTENT" | grep -qE "$PAT"; then
    echo "Refusing to write apparent secret. Pattern matched: $PAT" >&2
    echo "If this is a false positive, edit the file manually outside Claude Code." >&2
    exit 2
  fi
done

exit 0
```

> (脚本含义:从 JSON 输入里抽出 `content`(或 `new_string`,视工具而定),遍历一组"秘密数据"模式;命中任一就向标准错误写"Refusing to write apparent secret ..."及一条"若是误请在 Claude Code 外手动改"的指引,并以退出码 2 退出;否则以退出码 0 放行。其中:AKIA 开头 16 字符大写字母是 AWS access key ID; `ghp_` 开头 36+ 字符是 GitHub personal access token; `sk-` 开头 32+ 字符是 OpenAI API key 风格;PEM 私钥头部 `BEGIN ... PRIVATE KEY`;以及形如 `aws_secret_access_key=...` 的赋值行。)

← 前一页

后一页 →

> The script reads the content the tool is about to write, checks it against patterns that match well-known secret formats, and refuses the write if any match. The refusal includes a hint about how to bypass the check legitimately, by writing the file outside Claude Code. This kind of guidance in the refusal message is a small but valuable touch; it preserves the developer's ability to handle edge cases without making her wonder why the hook fired.

这段脚本读出"工具即将写入"的内容,对照一组匹配"知名秘密数据格式"的正则去查,任一命中就拒绝这次写入。拒绝消息里附带了一条"如果这是误报,就在 Claude Code 之外手动改"的提示。把这种"如何合理地绕过"的小提示写进拒绝消息,看似小事,实则很贴心——它

保留了开发者处理边界情况的能力,又不会让她摸不着头脑地想"这条 hook 怎么又跳出来了"。

Refusing Force Pushes · 拦截 Force-Push

> Move to force-push prevention. This hook addresses a specific category of git mistake that costs teams real time when it happens. A force push to main or to any protected branch can erase work done by other team members. The hook below intercepts any `git push` command that includes the force flag and refuses if the target branch is one of the protected ones.

下一条,讲 force-push 拦截。这条 hook 解决的是"一旦真发生,会真的让团队痛一阵子"的那一类 git 失误。对 main 或任何受保护分支的 force-push,都可能抹掉团队其他成员的工作。下面这条 hook 拦截任何带 force 标志的 `git push` 命令,若目标分支落在受保护清单里,就拒绝。

```
#!/usr/bin/env bash
# block-force-push.sh — refuses force pushes to protected branches.
CMD=$(jq -r '.tool_input.command')

if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease|-f[[:space:]])'; then
  if echo "$CMD" | grep -qE '(origin|upstream)[[:space:]]+(main|master|develop|production|release/)'; then
    echo "Refusing force push to protected branch." >&2
    echo "Force pushes to main/master/develop/production are blocked." >&2
    exit 2
  fi
fi

exit 0
```

> (脚本含义:从 JSON 输入里抽出 Bash 命令,先匹配是否含 force 标志(--force / --force-with-lease / `-f` 后跟空白);若命中,再匹配 remote 与受保护分支对(`main` / `master` / `develop` / `production` / `release/...`);同时命中就向标准错误写拒绝消息并以退出码 2 退出,否则以退出码 0 放行。)

> The script catches the force-push pattern in any of its common forms and checks the target branch against a protected list. The list can be extended for projects with additional long-lived branches. The protection is conservative, blocking even `--force-with-lease`, because in many real cases the developer's

这段脚本会捕捉各种常见形态的 force-push 模式,并把目标分支对照一份受保护清单去查。这份清单可以根据项目里额外的长期分支去扩展。这套保护偏保守——它连 `--force-with-lease` 也不放过,因为在很多真实场景中,开发者的

← 前一页

后一页 →

> intent in using `--force-with-lease` is benign but the consequences when it goes wrong are still substantial; an explicit override path through manual command-line use outside Claude Code remains available.

用意本是无害的,但"一旦出错后果仍然不小"这件事不会因此改变;真要强推,得绕开 Claude Code 在命令行里手动执行——这条"显式 override"的路径始终是开放的。

> The final security hook is a permission-confirmation gate for high-impact operations. Some actions are not strictly forbidden but should never happen without explicit human acknowledgment. Deploying to production. Running database migrations. Sending mass emails. Each of these can be encoded as a gate that prompts the user for confirmation rather than refusing outright. The mechanism uses a `PreToolUse` hook that, when it matches a high-impact pattern, opens a desktop notification or a small terminal prompt asking the user to confirm. If the user confirms, the hook exits with code zero and the action proceeds. If the user declines or the prompt times out, the hook exits with code two and the action is blocked.

最后一条安全 hook,是"对高影响操作做权限确认"的 gate。有些动作并不"严格禁止",但绝不该在"没有显式人工确认"的情况下发生:部署到生产、跑数据库 migration、群发邮件。这些都可以被编码成"提示用户确认而非直接拒绝"的 gate。机制是:一条 `PreToolUse` hook,一旦匹配到"高影响"模式,就弹一个桌面通知或一小段终端提示,问用户"是否允许"。用户允许,hook 以退出码 0 退出,动作放行;用户拒绝或提示超时,hook 以退出码 2 退出,动作被拦。

> The implementation varies by operating system, because the notification mechanism differs. On macOS, the `osascript` command can produce a confirmation dialog that returns a result. On Linux, `zenity` or `kdialog` can do the same. On Windows, PowerShell can produce a confirmation dialog. The configuration below uses macOS `osascript` as an example, but the pattern translates directly to the other systems.

具体实现因操作系统而异——通知机制不同。macOS 上,`osascript` 命令可以弹一个会回送结果的确认对话框;Linux 上,`zenity` 或 `kdialog` 同样可以;Windows 上,PowerShell 可以做等价的确认对话框。下面这份配置以 macOS `osascript` 为例,但这套模式可以直接平移到其他系统。

```
#!/usr/bin/env bash
# confirm-deploy.sh — requires explicit confirmation before deploy commands.
CMD=$(jq -r '.tool_input.command')

# Patterns that require explicit confirmation.
PATTERNS=(
  'kubectl[[:space:]]+apply.*production'
  'terraform[[:space:]]+apply'
  'aws[[:space:]]+s3[[:space:]]+sync.*production'
  'flyway[[:space:]]+migrate'
  'alembic[[:space:]]+upgrade'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$CMD" | grep -qE "$PAT"; then
    RESULT=$(osascript -e "display dialog \"Claude Code is about to run a high-impact command:\n\n$CMD\n\nAllow it?\
    if echo "$RESULT" | grep -q "Allow"; then
      exit 0
    else
      echo "User declined high-impact command: $CMD" >&2
      exit 2
    fi
  fi
done

exit 0
```

> (脚本含义:从 JSON 输入里抽出 Bash 命令,在一组"高影响命令"模式(`kubectl apply ... production` 、 `terraform apply` 、 `aws s3 sync ... production` 、 `flyway migrate` 、 `alembic upgrade`)上挨个走一遍;命中任一就弹一个 macOS 对话框,把命令原文摆出来,让用户在

"Cancel" / "Allow" 之间二选一;选 "Allow" 则退出码 0 放行,选 "Cancel"(或超时)则向标准错误写"User declined high-impact command: ..."并以退出码 2 退出。)

← 前一页 后一页 →

The script reads the Bash command from the JSON input, checks it against a list of patterns that match high-impact operations, and if any match, opens a system dialog asking the user whether to allow the command. The dialog is modal; the script blocks until the user responds. If the user clicks `Allow`, the script exits with code zero and the command proceeds. If the user clicks `Cancel` or dismisses the dialog, the script exits with code two and the command is blocked. The user is now in the loop for any operation that touches production, with no possibility of a slip-up where the model decides on its own that a deployment is safe to run.

这段脚本从 JSON 输入里读出 Bash 命令,与一份"高影响操作"模式清单逐项比对,任一命中就弹出一个系统对话框,问用户"是否允许这条命令"。这个对话框是模态的——脚本会在用户回应之前一直阻塞。用户点 `Allow`,脚本以退出码 0 退出,命令继续;用户点 `Cancel` 或直接关掉对话框,脚本以退出码 2 退出,命令被拦。开发者由此被嵌入"任何触及生产的操作"的人在回路,杜绝了"模型自顾自判断某次部署可以安全跑"这种事。

The list of patterns in the confirmation gate is, like the lists in the earlier hooks, intended to be edited as the developer learns. New high-impact operations will surface over time, and each one should be added to the list as it is recognized. The hook is most valuable when its pattern list is comprehensive, because every gap is an operation that could happen without confirmation, and confirmation gates are most useful when they leave no significant gaps.

和前面那些 hook 一样,这份确认 gate 的模式清单也是"边学边改"的——新的高影响操作会随着时间浮现,每识别出一个,就应当补进清单。模式清单越齐全,这条 hook 越有价值;因为每一道缺口,都是一次"可能绕开确认"的潜在操作,而确认 gate 最有用的地方,恰恰是它"没留下什么明显缺口"。

The Threat Model · 威胁模型

A reflection on the threat model is worth including, because it clarifies what the hooks in this chapter do and do not protect against. The hooks protect against accidental damage caused by Claude itself or by the developer through Claude. They prevent Claude from running an `rm -rf` when the developer asked for help cleaning up files. They prevent Claude from writing a file to `.env` when the developer asked for help with environment variables. They prevent Claude from force-pushing to main when the developer asked for help reorganizing branches. The protection is against the failure mode where the developer's broad request gets translated by the model into an action she did not intend.

关于"威胁模型",值得专门说几句——它能讲清本章的 hook "在防什么、不在防什么"。这些 hook 防的是"由 Claude 自己,或经由 Claude 由开发者触发"的那一类意外损害。它们阻止 Claude 在"开发者只是请它帮忙整理文件"时去跑 `rm -rf`;阻止 Claude 在"开发者只是请它帮忙处理环境变量"时把内容写进 `.env`;阻止 Claude 在"开发者只是请它帮忙整理分支"时去 force-push 到 main。整道防线防的是这样一种失效模式:开发者给出一个比较宽泛的请求,被模型翻译成了她本不打算做的那个具体动作。

← 前一页 后一页 →

> The hooks do not protect against a deliberately malicious user. A user with shell access to the developer's machine can edit the hook scripts, disable the hook configuration, or simply run the dangerous command directly outside Claude Code. The hooks are not a security boundary against an adversary. They are a safety boundary against accident, and that is exactly the boundary they are designed to be. A security architect who reads this chapter and concludes that hooks are insufficient as a complete security solution is correct. The right framing is that hooks are one layer in a defense-in-depth model, and the layer they occupy is the one closest to the developer, where most actual incidents in practice originate. The other layers, including operating system permissions, version control protections, and infrastructure access controls, remain necessary, and the hooks complement them rather than replacing them.

这些 hook 防不住"主动恶意的用户"。拿到这位开发者机器 shell 权限的人,可以直接改 hook 脚本、关掉 hook 配置,或者干脆绕开 Claude Code 在终端里直接跑那条危险命令。Hook 不是面向"敌手"的安全边界;它是面向"意外"的安全边界——而这恰恰是它设计要成为的那道边界。一位安全架构师读完本章,得出"hook 不足以构成完整的安全方案"这一结论,是正确的。更恰当的框架是:hook 是"纵深防御"模型里的一层,它占据的那一层,正是"最贴近开发者"的那一层——而绝大多数真实事故,正是在这一层发生的。其他层(操作系统权限、版本控制保护、基础设施访问控制)依然必要;hook 补足它们,而不是取代它们。

> Together, the five hooks in this chapter form a security-hook bundle that addresses the most common categories of risk a developer faces in a Claude Code session. The hooks are not exhaustive; security is a continuous practice, not a fixed checklist, and the developer should expect to add to her bundle as her work surfaces new risks. But the five above cover the largest categories at the lowest cost. Installing them takes roughly an hour. The protection they provide compounds over every future session, and the cost of not having them, even once, is potentially substantial.

这五条 hook 合在一起,就构成了一套"安全 hook 套件",覆盖了 Claude Code session 中开发者最常碰到的那些风险类别。它们并不穷尽一切——安全是一项持续实践,不是一份固定清单;随着工作中浮现新的风险,开发者应当预期会持续往里加。但这五条已经以最低的成本覆盖了最大头的几类。装好它们大约要一小时;它们提供的保护,会在之后每一个 session 里持续复利;而"某一次没装"的代价,哪怕只一次,都可能大得惊人。

> A second reflection concerns the experience of working with these hooks installed. The hooks are quiet most of the time. They sit in their gate positions, evaluating every relevant tool call, and they almost never block, because the developer almost never asks Claude to do anything dangerous. The cost of their presence is negligible. Then, one day, a hook fires. The developer asked for something that, on careful reading, would have led Claude to write a secret to a file or to run a command she did not actually intend. The hook stops the action. The session continues without damage. The developer realizes, in retrospect, that she had asked for something she did not mean. The hook just bought her a substantial amount of cleanup time, or worse. This pattern, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even before any incident has demonstrated their value.

第二条反思,关于"装上这些 hook 之后的工作体感"。Hook 大多数时候都安安静静——它们守在 gate 的位置上,对每一条相关工具调用做评估,而几乎从不拦截,因为开发者几乎从来不会让 Claude 去干危险的事。它们常驻的代价几乎为零。然后某一天,某条 hook 跳了出来——开发者提了一个请求,仔细一读,本来会让 Claude 把一段秘密写进文件,或去跑一条她并不真打算跑的命令。Hook 拦下了这个动作,session 没有受到任何损害;开发者事后回看,意识到自己方才开口要的,其实并不是自己真正想做的。这条 hook 在那一刻,替她省下了一大笔收尾的时间——或者更糟。这个模式——"日常成本极低,单次事件回报极高"——正是安全 hook 在"还没出过任何事故来证明它的价值"之时,就值得装上的原因。

> A fourth observation, more practical, concerns the maintenance of the security bundle over time. The patterns the hooks check for will need to

第四条观察更偏实操,关于"安全套件随时间的维护"。Hook 所检查的那些模式,要随着开发者的工作一起演化:新的危险命令会被发现;新的受保护路径会随项目生长而变得相关;新的秘密格式会随着接入新的服务、用上新的 key 形状而出现。Hook 套件不是"装完就忘"的一次性安装——它是一份"小但真实"的持续维护;开发者应当每几个月回头看一遍,往清单里加新模式、移掉不再适用的旧模式。每季度花十

五分钟维护,就能让套件保持有效;而一旦把 hook 脚本写干净、让模式清单清楚地放在每个文件的最上面,维护起来就轻松得多。钩子写得"易于维护"这条纪律,归根结底,决定了这套件是多年里一直好用,还是悄无声息地漂成无用之物。

← 前一页

后一页 →

> evolve as the developer's work evolves. New dangerous commands will be discovered. New protected paths will become relevant as projects grow. New secret formats will emerge as the developer adopts new services that use new key shapes. The hook bundle is not a fire-and-forget installation. It is a small but real piece of ongoing maintenance, and the developer should plan to revisit it every few months, adding new patterns to the lists and removing patterns that no longer apply. Fifteen minutes of maintenance per quarter is sufficient to keep the bundle effective, and the maintenance is much easier to perform if the hook scripts are written cleanly with their pattern lists clearly visible at the top of each file. The discipline of writing hooks that are easy to maintain is, in the end, the discipline that determines whether the bundle remains useful over the years or quietly drifts into uselessness.

evolve as the developer's work evolves. New dangerous commands will be discovered. New protected paths will become relevant as projects grow. New secret formats will emerge as the developer adopts new services that use new key shapes. The hook bundle is not a fire-and-forget installation. It is a small but real piece of ongoing maintenance, and the developer should plan to revisit it every few months, adding new patterns to the lists and removing patterns that no longer apply. Fifteen minutes of maintenance per quarter is sufficient to keep the bundle effective, and the maintenance is much easier to perform if the hook scripts are written cleanly with their pattern lists clearly visible at the top of each file. The discipline of writing hooks that are easy to maintain is, in the end, the discipline that determines whether the bundle remains useful over the years or quietly drifts into uselessness.

The Team Dimension · 团队维度

> A fifth observation concerns the team dimension of the security bundle. Security hooks are valuable when one developer installs them, and they are dramatically more valuable when an entire team installs the same set across all of its projects. A team that has agreed on a standard security bundle, encoded in shared configuration that travels with each project, raises the floor of safety for everyone on the team simultaneously. New team members inherit the bundle automatically the moment they clone any project. The team's collective risk of an incident drops sharply, because the most common categories of accident are blocked across every developer's environment, not just in the environments of the developers who happened to install hooks personally. This is the team-level argument for hooks, and it is the argument that justifies treating the security bundle as a shared engineering asset rather than as a personal preference. Teams that adopt this view tend to have strikingly fewer incidents than teams that leave each developer to her own preferences, and the difference is not subtle.

第五条观察,关于安全套件的"团队维度"。单个开发者装上安全 hook,是有价值的;而当整个团队在所有项目上都装上同一套,价值会被大幅放大。一个团队若就"标准安全套件"达成共识,并把它编码进随每个项目走的共享配置里,就能同时抬高全员的安全底线。新成员一克隆任何项目,套件就自动继承到手;团队的事故总风险会急剧下降——因为最常见的那几类意外,在每一位开发者的环境里都已被拦下,而不是只落在"碰巧个人装了 hook"的那几位开发者身上。这是"hook 必要性"的团队级论据,也正是它把"安全套件"从"个人偏好"提升为"团队工程资产"的那条理由。秉持这种看法的团队,通常比"把偏好留给每个人自己"的团队,事故少得多——这份差异,可不是"细微"。

> A sixth and final observation concerns the relationship between security hooks and the other two pillars of the framework. Skills and subagents both shape what Claude does within the agent loop, and both can encode warnings about dangerous operations. A skill could include a directive

to *never* run `rm -rf`, and a subagent system prompt could include a similar warning. These shapings are useful, but they are not enough on their own,

第六条、也是最后一条观察,关于"安全 hook"与框架另外两根支柱之间的关系。Skill 与 subagent,都塑造 Claude 在 agent loop 内部的行为,也都可以把"危险操作的警告"编码在内。一条 skill 可以包含"绝对*不要*跑 `rm -rf`"这种指令,一个 subagent 的 system prompt 也可以放一段类似的警告。这些塑造是有用的,但单靠它们并不够——

← 前一页

后一页 →

because they ride on the model's probabilistic compliance with its instructions. The security hook is what turns the warning into a guarantee. The right architectural pattern is to have all three: a skill that explains what good practice looks like, a subagent that operates within that practice, and a hook that enforces the boundary that even the best skill and subagent cannot guarantee. The three layers reinforce each other. The skill teaches. The subagent applies. The hook enforces. The combination is stronger than any one layer alone, and developers who think of security as a single-layer concern miss the architectural elegance that the three-pillar approach provides.

因为它们都"骑"在模型对其指令的概率性遵循之上。安全 hook,才是把"警告"变成"保证"的那一层。正确的架构模式,是三层都要有:一条 skill 把"好的实践是什么样的"讲清楚;一个 subagent 在那条实践里做事;一条 hook 把边界强制住——这条边界,即便再好的 skill 和 subagent 也无法保证自己不去踩。三层彼此增益:skill 教;subagent 照做;hook 强制。组合起来,比任一层单兵作战都要强;把"安全"当成"单层问题"看的开发者,会错过三支柱方法在架构上的那份优雅。

A third reflection concerns the relationship between hooks and trust. Without hooks, the developer has to trust the model on every action. Trust, in this context, means hoping that the model will interpret her requests correctly and will not produce dangerous side effects. Trust is appropriate for many things, but it is not appropriate for catastrophic failure modes, because the failure rate of any probabilistic system is non-zero, and a non-zero failure rate over many actions eventually produces a failure. Hooks change the relationship. With hooks, the developer trusts the model for most things and relies on mechanism for the things that matter. This is a better division of labor than asking the model to be perfect, because perfection is not what models are good at, and mechanism is exactly what mechanism is good at.

第三条反思,关于 hook 与"信任"的关系。没有 hook,开发者就只能在每一次动作上信任模型。这里的"信任",其实质是"指望着模型正确解读她的请求,且不产生危险的副作用"。对很多事,这种信任是恰当的;但对"灾难性失效"那一类,这种信任并不恰当——任何概率性系统,其失败率都不为 0;而"不为 0 的失败率"摊在足够多的动作上,迟早会引发一次失败。Hook 改变了这种关系:有了 hook,开发者把多数事交给模型去信任,把"真正要紧的事"交给机制去兜底。这比"要求模型完美"是更好的分工——因为"完美"本就不是模型所擅长的;而"机制",恰恰是机制所擅长的。

A reflection before this chapter closes. Security hooks have a quiet quality that distinguishes them from skills and subagents. Skills produce visible value: every commit message, every PR description, every refactor proposal is something the developer sees and uses. Subagents produce visible value too: every parallel review, every specialized output is a tangible artifact. Security hooks produce value that is mostly invisible. Most days, no security hook fires, because the developer is not asking Claude to do anything dangerous. The hooks sit silently. Their value reveals itself only on the day when something goes wrong and a hook stops it from going more wrong. That day, the value of the entire bundle is realized in a single block. The asymmetry, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even if they never fire. The developer who waits until something goes wrong to install them has discovered the value of hooks at the worst possible moment. The

本章收束前的反思:安全 hook 有一种"安静"的特质,把它与 skill、subagent 区分开。Skill 产出的是可见的价值——每一条 commit message、每一条 PR 描述、每一份重构建议,都是开发者看得见、用得上的东西。Subagent 也产出可见的价值——每一次并行评审、每一份专家产物,都是有形有质的工件。安全 hook 产出的价值,几乎全在暗处。大多数日子里,没有安全 hook 跳出来——因为开发者根本没让 Claude 去干危险的事。Hook 们安安静静地蹲着;它们的价值,只在某天"出了岔子、而某条 hook 把更大的岔子拦下"那一刻,才浮出水面。那一天,整套装件的价值,以一次拦截兑现。这种"日常成本极低、单次事件回报极高"的不对称,正是安全 hook 哪怕一次都没真正触发,也仍然值得装上的原因。等到"出了事再装"的开发者,是在最不该的时机才看清 hook 的价值。

← 前一页

后一页 →

> developer who installs them as part of her standard environment has bought insurance she will probably never use, which is exactly the kind of insurance worth having.

把安全 hook 作为标准环境的一部分装上的开发者,买了一份自己大概永远用不到的"保险"——而这恰恰是"值得买的保险"的本意。

> The next chapter takes hooks one more step into composition. With single hooks installed, the question becomes how multiple hooks work together, how event pipelines emerge when hooks chain, and how the developer can construct sophisticated lifecycle workflows out of the same simple primitives. Hook chains are the final piece of hook design, and they are the subject of the chapter ahead.

下一章,把 hook 再推一步,推入"组合"。单条 hook 装好之后,真正的问题变成:多条 hook 如何协作?hook 链化时,事件流水线如何浮现?开发者又如何用同一批简单的原语,搭出精细的生命周期工作流?Hook 链是 hook 设计最后一块拼图,也是下一章要讲的事。

← 前一页

Chapter 18 (Hook Chains) →

CHAPTER EIGHTEEN · 第十八章
Hook Chains and Event Pipelines · Hook 链与事件流水线

> *"A pipeline is what happens when many small actions, each useful in itself, are arranged so that the output of one becomes the context of the next."*
>
> — Doug McIlroy, paraphrased

"所谓流水线,就是把许多本身各自有用的小动作,安排成'上一个的输出,成为下一个的上下文'。"

—— 道格·麦克罗伊(意译)

This chapter is the final treatment of hooks before the book turns to composition across all three pillars. The previous three chapters built hooks one at a time: the mechanics, the design patterns, the security applications. This chapter puts hooks together. When multiple hooks fire on the same event, when hooks pass information forward through the lifecycle, when the developer wants a coordinated workflow that involves several lifecycle moments rather than just one, the resulting structure is a hook chain or an event pipeline. This chapter teaches both.

本章是关于 hook 的最后一章,之后书就要谈"三根支柱如何彼此组合"了。前三章,是一根一根地讲 hook:机制、设计模式、安全应用。本章要把 hook 装到一起。多条 hook 在同一事件上触发、hook 在生命周期里把信息往前传、开发者想要的不止一个生命周期节点而是几个——这些情况所衍生出来的结构,就是 hook 链或事件流水线。本章把两者都讲一遍。

Begin with the simpler case. A hook chain is what happens when more than one hook is configured under the same event with the same matcher. Claude Code runs them in the order they are listed in the configuration file. Each hook receives the same input on standard input. Each hook's exit code is independent: one hook's decision to allow or block does not affect what the next hook does. The chain proceeds through all hooks unless one of them blocks, in which case the action is blocked and subsequent hooks in the same chain do not run. The output of each hook is collected, but the hooks themselves do not communicate directly with each other. Communication, when it is wanted, happens through side effects on the file system or through environment variables.

从简单的那种情形讲起。Hook 链,就是"同一个事件、同一个 matcher 下配了不止一条 hook"时发生的事。Claude Code 按配置文件中列出的顺序执行它们。每条 hook 都在标准输入上收到同样的输入;每条 hook 的退出码彼此独立——一条 hook 的"放行/拦截"决定,不影响下一条 hook。链会一路走完整段,除非其中某条拦下,这时动作被拦、链上后续的 hook 不再跑。每条 hook 的输出会被收集,但 hook 彼此之间并不直接通信。需要通信的时候,就通过文件系统的副作用,或通过环境变量。

The Prettier-and-ESLint example from Chapter Sixteen is a small chain. Two hooks fire on the same `PostToolUse` event with the same matcher. The first runs Prettier; the second runs ESLint. The order matters in this case, because Prettier reformats the file and ESLint then applies its fixes on the reformatted version. If the order were reversed, the result would be subtly different in some edge cases. The developer who designs a chain should think about whether the order of hooks is meaningful, and if it is, she should comment the configuration to make the order explicit for future readers.

第十六章的 Prettier + ESLint 例子,就是一条小链:两条 hook 在同一个 `PostToolUse` 事件、同一个 matcher 下触发。第一条跑 Prettier,第二条跑 ESLint。这里顺序重要——Prettier 把文件重新格式化,ESLint 再在格式化后的版本上施加它的修复;若顺序反过来,某些边界情形会得到微妙不同的结果。设计 hook 链的开发者,应当想清楚"顺序是否有意义";若有,就该在配置里加注释,让顺序对未来的读者也是显式的。

← 前一页

后一页 →

A second example, slightly more elaborate, shows a chain that combines a notifier with a guard. The configuration runs after every `Bash` command that Claude executes: it logs the command to an audit file, then checks whether the command has produced any new git changes, and if so, writes a small note to a session-summary file that the developer can review at the end of the day.

第二个例子稍复杂一点,展示一条把 notifier 与 guard 组合起来的链。配置在 Claude 跑完每一条 `Bash` 命令后触发:先把命令写进审计文件,再检查该命令是否产生了新的 git 变更;若有,就在一份"当日 session 摘要"文件里写一条小备注,供开发者晚些时候回看。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '\"\\(.session_id)\\t\\(.tool_input.command)\"' >> ~/.claude/bash-history.log"
          },
          {
            "type": "command",
            "command": "if git status --porcelain 2>/dev/null | grep -q . ; then jq -r '.tool_input.command' >> ~/.c
          }
        ]
      }
    ]
  }
}
```

(配置含义:在 PostToolUse 上挂两条 hook 形成的链,matcher 限定 Bash 工具;第一条以制表符分隔的 " \t " 格式追加到 `~/.claude/bash-history.log` ;第二条检查 `git status --porcelain` 若有变更,就把该命令追加到 `~/.claude/session-summary.txt` 。)

Two Hooks in Sequence · 两条 hook 串行

The two hooks fire in sequence on the same event. The first appends a tab-separated record to the audit log. The second checks the git status and, if changes are present, appends the command to a session summary. Both run on every Bash call. Both are silent in the success case. Together, they produce an audit trail and a daily summary without any conscious intervention from the developer.

这两条 hook 在同一事件上串行触发。第一条向审计日志追加一条 tab 分隔的记录;第二条检查 git 状态,若有变更就把命令追加到 session 摘要。两者在每次 Bash 调用时都跑;两者在成功路径上都沉默。合起来,就能在开发者完全无须主动介入的情况下,产出一份审计轨迹和一份当日摘要。

Move to event pipelines, which is the more sophisticated case. An event pipeline is a workflow that spans multiple lifecycle events, with hooks at each event reinforcing or extending the others. The classic example is a git pipeline: a `SessionStart` hook ensures the working tree is clean and reports the

进入事件流水线——这是更复杂的那一种情形。事件流水线,是一条跨越多个生命周期事件的工作流;每个事件上都挂着 hook,彼此强化、彼此延展。经典例子就是一条 git 流水线: `SessionStart` hook 确保工作树是干净的,并汇报

← 前一页

后一页 →

current branch, a `PreToolUse` hook on Bash blocks force-pushes to main, a `PostToolUse` hook on Edit or Write runs the formatter, a `Stop` hook checks whether tests pass before allowing the session to end with uncommitted changes. None of these hooks knows about the others

> directly. But together they form a pipeline that wraps every git-related action in a consistent set of guarantees, and the developer who has installed the pipeline experiences her git work differently because every step is now defended in the right place.

当前分支;Bash 上的 `PreToolUse` hook 拦截 force-push 到 main;Edit/Write 上的 `PostToolUse` hook 跑格式化; `Stop` hook 在 session 结束前检查测试是否通过,以免"session 结束时还存在未提交的变更"。这些 hook 彼此并不直接"知道"对方的存在;但合在一起,它们构成了一条流水线,把每一项与 git 相关的动作,都包进同一套一致的保证里。装上这条流水线的开发者,她的 git 工作会"感觉不一样"——每一步都恰好守在它该被守的地方。

> A worked event pipeline is presented below. The configuration is more substantial than any in the previous chapters, because it is doing real cross-event coordination. The pipeline assumes a project where tests are run with `npm test`, the main branch is named `main`, and the developer wants Claude Code to be especially careful with anything that touches that branch.

下面是一条完整的事件流水线。配置比前几章任何一条都更厚实——它在跨事件做实在的协调。这条流水线假定:测试用 `npm test` 跑、主分支名为 `main`、开发者希望 Claude Code 对"触及主分支"这件事格外小心。

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --short"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
          },
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-force-push.sh"
          }
        ]
      },
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
```

```
            "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\" 2>/dev/null || true"
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "if [ \"$(git status --porcelain | wc -l)\" -gt 0 ]; then echo '{\"decision\":\"block\",\"rea
          }
        ]
      }
    ]
  }
}
```

(配置含义:这是一个跨四个生命周期事件的完整 git 流水线。SessionStart:打印当前分支和 `git status --short`,注入到 Claude 的启动上下文;PreToolUse(Bash):同时挂 `block-dangerous.sh` 和 `block-force-push.sh` 两条 gate;PreToolUse(Edit|Write|MultiEdit):挂 `protect-files.sh` 一条 gate;PostToolUse(Edit|Write|MultiEdit):跑 Prettier 格式化;Stop:若有未提交变更,向 stdout 打 JSON 决策块 `{"decision":"block","reason":"..."}`,让模型无法在带未提交变更的情况下结束 session。)

← 前一页

后一页 →

Read the configuration carefully, because it covers four lifecycle events and demonstrates how a serious pipeline holds together. `SessionStart` runs once when the session begins and prints the current branch and any uncommitted changes; the output is injected into Claude's starting context, so Claude knows the state of the world at the start of every session. `PreToolUse` on `Bash` runs the dangerous-command and force-push gates from Chapter Seventeen. `PreToolUse` on `Edit` and `Write` runs the protected-files gate. `PostToolUse` on `Edit` and `Write` runs Prettier. `Stop` runs a check that uses the JSON output format to refuse the session's end if there are uncommitted

请把这份配置仔细读一遍——它覆盖了四个生命周期事件,示范了一条严肃的流水线是怎么"粘合"起来的。`SessionStart` 在 session 启动时跑一次,打印当前分支以及任何未提交的变更;输出会被注入到 Claude 的启动上下文里,因此 Claude 在每个 session 一开始就"知道外面世界长什么样"。`PreToolUse` 在 `Bash` 上挂第十七章讲过的"危险命令"与"force-push"两条 gate。`PreToolUse` 在 `Edit` / `Write` 上挂"受保护文件"的 gate。`PostToolUse` 在 `Edit` / `Write` 上跑 Prettier。`Stop` 做一项"用 JSON 输出格式"做判定:若存在未提交

changes; the JSON output format is what allows the Stop event to actually refuse the end of the session, because the standard exit-code mechanism cannot stop a session that is already ending. The pipeline as a whole ensures that: Claude always knows the branch and clean-status state of the world; dangerous shell commands are blocked; force pushes to main are blocked; protected files cannot be edited; every file is formatted; and the session cannot end with uncommitted changes. The combination is a substantial improvement over the unconfigured environment, and it costs about a hundred lines of JSON to install.

变更,就拒绝 session 结束。这里之所以要"用 JSON 输出格式"——是因为 Stop 事件对应的"会话即将结束"这件事,光靠"退出码"那条机制拦不住,只有 JSON 决策块才拦得住。这条流水线作为整体,保证了:Claude 总能知道当前分支和 clean-status 状态;危险的 shell 命令

被拦;force-push 到 main 被拦;受保护文件改不了;每一个文件都被格式化;session 不会在"带未提交变更"的状态下结束。这套组合,比未配置的环境是一个显著升级;而装好它总共才百来行 JSON。

The Power of Coordination · 协调的力量

> The key insight from this pipeline is the coordination across events. None of the individual hooks is doing something the developer could not do manually. She could, in principle, run Prettier by hand after every Claude edit, run `git status` by hand before every commit, refuse to allow force pushes by typing the wrong command into her own terminal. The point of the pipeline is not that any individual hook is doing something new. The point is that the developer no longer has to remember to do any of it. The pipeline remembers for her. The cumulative effect of having all these little guarantees enforced mechanically, rather than remembered, is that the developer's attention is freed for the parts of the work that genuinely require attention. The mechanical parts run mechanically. The interesting parts get the focus they deserve.

这条流水线带出的关键洞见,是"跨事件协调"本身。任一单条 hook 都没有做开发者"手动做不了"的事——她原则上可以每次 Claude 改完手动跑一次 Prettier、提交前手动跑一次 `git status`、想 force-push 时自己在终端里敲错命令拒绝自己。流水线真正的意义,并不在"哪一条 hook 做了什么新鲜事";而在于"开发者再也不必记得去做这些"。流水线替她记着。所有的这些"小事"由机制强制而不是靠记忆——其累积效果是:开发者的注意力,被释放出来,留给那些真正需要她注意的工作。机械的部分,机械地跑;有意思的部分,得到它应得的专注。

← 前一页

后一页 →

> changes, returning a decision of `block` with a clear reason that surfaces to Claude.

变更,就以 `block` 决策回送,并附上一段清晰的 reason,送达 Claude。

A Coherent Workflow · 一条内洽的工作流

> The pipeline together enforces a coherent workflow. Sessions begin with awareness of state. Tool calls are filtered through gates appropriate to their kind. Edits are followed by formatting. Sessions cannot end with the working tree dirty. The developer has not had to remember any of these rules, because each rule is enforced by a hook at the right lifecycle moment. The development experience is, in a sense, supervised, but the supervision is mechanical and silent, and it intervenes only when the developer was about to do something the pipeline considers a mistake.

这条流水线,作为一个整体,强制的是一条内洽的工作流:session 启动时即知"世界状态";工具调用按各自类型被相应的 gate 过滤;编辑后即格式化;session 不会在 dirty 工作树的状态下收尾。开发者不再需要记得这些规则中的任何一条——每条规则都被一个 hook 守在它"该被守的"那个生命周期节点上。从某种意义上,这种开发体验是被"监督"着的;但这种监督是机械的、沉默的,只在开发者正要做"流水线认为是个错"的某件事时,才插一手。

> A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has not earned its place after a month of use. The pipeline

should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

关于事件流水线,有一条值得直接点出的纪律:流水线很强,但它同时也是"累积复杂度"的一种形态;流水线里每条 hook,都是开发者承诺要维护的。一条三四条 hook 精心挑选的小流水线,比"完全没有流水线"已是显著升级;而十五条 hook 的大流水线,可能反而不如七条——因为失效模式成倍增长,累计延迟叠加,多 hook 共同运行时开发者 debug 异常行为的能力也会退化。推荐做法是:小起步、一次加一条、用了一个月还没证明自身价值的 hook 就拿掉。流水线应当服务于开发者,而不是反过来;一条"长成了维护负担"的流水线,就是被过度工程化的流水线。

A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

第二条纪律关于测试。每条 hook 都应当在"提交"前测过。标准测试模式:在 shell 里直接调用它,在标准输入上喂一段"有代表性"的 JSON 输入,然后观察它的退出码和标准错误输出。在直接测试下行为正确的 hook,通常在 Claude Code 内也会正确;而直接测试下都不对的 hook,进了 session 大概率会吐出莫名其妙的失败。安装时花的这十五分钟直测,免去了日后的一整轮 debug。这套模式,任何"做的不止一行命令"的事,都值得无条件采用。

A second worked pipeline is worth presenting, because pipeline design is the kind of skill that matures only after a developer has seen several

第二条完整流水线值得摆出来——因为流水线设计这门手艺,只在看过若干条

← 前一页

后一页 →

substantial examples. The pipeline below is for a different shape of work: a content-generation project, where the developer is using Claude Code to produce written material rather than to write code. The pipeline reflects the different concerns of writing work. Drafts should be backed up automatically. Drafts should be passed through a humanizer pass after every edit. Sessions should end with a daily summary appended to a project journal. The pipeline below implements each of these.

比较"实在"的例子之后,才会真正成熟。下面这条流水线服务的,是一种不同形态的工作——一个"内容生成"项目,开发者用 Claude Code 产出的不是代码而是书面材料。流水线反映了写作这件事的关切:草稿应当自动备份;每次编辑之后,应当过一次"人味化"pass;session 结束时,应当向项目日志追加一份当日小结。下面这条流水线,把每一项都实现了。

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '## Today, you are working on:'; cat $CLAUDE_PROJECT_DIR/.claude/today.txt 2>/dev/null
```

```
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); cp \"$FILE\" \"$CLAUDE_PROJECT_DIR/.backups/$(basenam
            "async": true
          },
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); if echo \"$FILE\" | grep -qE '\\.(md|txt)$'; then $CL
          }
        ]
      }
    ],
    "SessionEnd": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo \"$(date '+%Y-%m-%d %H:%M') session ended. Files touched today:\" >> $CLAUDE_PROJECT_DI
          }
        ]
      }
    ]
  }
}
```

(配置含义:这是一条"内容生成"项目的事件流水线。SessionStart:读 `.claude/today.txt` 焦点文件,把它注入到 Claude 的启动上下文;若文件不存在则给一条提示。PostToolUse(Edit|Write):挂两条 hook——第一条以 `async: true` 异步地把被编辑的文件备份到 `.backups/<文件名>.<时间戳>` ,不影响 session 速度;第二条仅在文件后缀为 `.md / .txt` 时,调用 `check-em-dashes.sh` 检查"em-dash"等被禁用的写作习惯。SessionEnd:向项目 `journal.md` 追加一行"session 结束"加时间戳,再附上近 5 次提交里被改过的文件名清单。)

← 前一页

后一页 →

Read the pipeline carefully, because its shape is informative. `SessionStart` loads a focus file, which the developer has placed in the project to remind herself what she is working on today. The output is injected into Claude's starting context, so Claude knows the day's focus from the moment the session begins. `PostToolUse` fires two hooks on every `Edit` or `Write` . The first is asynchronous: it backs up the file to a timestamped copy in a backups directory, without slowing down the session. The second runs only on Markdown or text files, and it invokes a custom script that checks for em-dashes and other patterns the developer wants to avoid in her writing. `SessionEnd` appends a record to a project journal, giving the developer a continuous log of what she worked on across sessions. The pipeline does not block any of these actions. It does not interfere with Claude's flow. It quietly produces backups, applies the writing standard, and maintains the journal, all without conscious intervention.

请把这条流水线仔细读一遍——它的形状本身就是有信息量的。`SessionStart` 加载一份"焦点文件",这是开发者放在项目里、用来提醒自己"今天做什么"的;其输出会被注入到 Claude 的启动上下文,于是 Claude 在 session 一开始就掌握了"今日焦点"。`PostToolUse` 在每次 `Edit` / `Write` 上挂两条 hook——第一条是异步的:它把文件复制成一份带时间戳的备份,放进 backups 目录,并不拖慢 session;第二条仅对后缀为 Markdown 或 txt 的文件触发,调用一个自定义脚本,检查 em-dash 以及其他"开发者希望在自己的写作里回避"的模式。`SessionEnd` 向项目日志追加一条记录,给开发者留下跨 session 的一份"自己今天做了什么"的连续流水。流水线不拦截任何动作,不打断 Claude 的节奏,只是悄悄产出备份、应用写作规约、维持日志——一切无须开发者主动介入。

What Pipelines Buy You · 流水线真正"买到"的是什么

> The pipeline is small, but it changes the developer's experience of working on the project substantially. She knows that backups exist. She knows that her em-dash policy is being enforced on every edit. She knows that her journal is up to date. The mental load of remembering these things has shifted from her to the pipeline, and the load that has been removed from her is now available for the actual writing work. This is the deeper benefit of pipelines: they take ambient concerns off the developer's mind and handle them as standing background processes, freeing the developer's attention for the work that requires attention.

这条流水线虽小,却显著地改变了开发者在该项目上的工作体验:她知道有备份存在;她知道每次编辑 em-dash 规约都在生效;她知道日志是同步的。"记得这些事"的心智负担,从她身上挪到了流水线上;而腾出来的那份注意力,如今可以投到真正的写作工作上。这正是流水线更深一层的收益——它把"背景里的事"从开发者脑子里搬走,当成"长期运行的后台进程"去处理,把开发者的注意力释放给那些"真正需要注意力"的工作。

> A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to

关于事件流水线,有一条值得直接点出的纪律:流水线很强,但它同时也是"累积复杂度"的一种形态;流水线里的每一条 hook,都是开发者承诺要去维护的——

← 前一页

后一页 →

> maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has not earned its place after a month of use. The pipeline should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

一份三四条 hook 精心挑选的小流水线,比"完全没有流水线"已是显著升级;而十五条 hook 的大流水线,可能反而不如七条——因为失效模式成倍增加,累计延迟叠加,多 hook 共同运行时开发者 debug 异常行为的能力也会退化。推荐做法是:小起步、一次加一条、用了一个月还没证明自身价值的 hook 就拿掉。流水线应当服务于开发者,而不是反过来;一条"长成了维护负担"的流水线,就是被过度工程化的流水线。

> A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A

> hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

第二条纪律关于测试。每条 hook 都应当在"提交"前测过。标准测试模式:在 shell 里直接调用它,在标准输入上喂一段"有代表性"的 JSON 输入,观察退出码和标准错误输出。直接测试下行为正确的 hook,通常在 Claude Code 内也正确;而直接测试下都不对的 hook,进了 session 大概率会吐出莫名其妙的失败。安装时花的十五分钟直测,免去日后的一整轮 debug。这套模式,任何"做的不止一行命令"的事,都值得无条件采用。

> A third discipline concerns version control. The `.claude` directory of a project, including the `settings.json` file that holds the hook configuration, should be version-controlled alongside the source code. Hooks are part of the project's contract for how work happens in the repository. A team member who clones the project for the first time should receive the entire pipeline automatically, without needing to configure anything herself. The version control discipline ensures this. Pipeline changes should be reviewed in pull requests like any other code change, because a hook change can affect every future session in the project, and the impact of a bad hook can be substantial. The same engineering hygiene that applies to source code applies to hook configuration, and projects that treat their `.claude` directory with this discipline end up with stable, predictable, well-understood pipelines that genuinely serve the team.

第三条纪律关于版本控制。项目的 `.claude` 目录(含那份装 hook 配置的 `settings.json`)应当与源码一起进版本控制。Hook 是项目"工作如何发生"这份契约的一部分。第一次 clone 项目的团队成员,应当自动拿到整条流水线,而不必自己再配置什么——版本控制纪律保证了这一点。流水线的变更,应当像其他代码变更一样走 PR 评审;因为一条 hook 的变更会作用于该项目里之后所有的 session,而一条"坏 hook"的影响可能非常显著。源码适用的那套工程卫生,同样适用于 hook 配置;以这种纪律对待自家 `.claude` 目录的项目,最终都会得到"稳定、可预测、被充分理解"的流水线,真正服务于团队。

Pipelines and Engineering Practice · 流水线与工程实践

> A fourth observation, more philosophical, concerns what pipelines mean for the practice of software engineering. The traditional way of enforcing

第四条观察更偏哲学,关于"流水线"对软件工程实践本身意味着什么。传统上,强制项目规约的方式是——

← 前一页

后一页 →

> project conventions has been documentation, training, and review. Documentation tells people what the conventions are. Training builds the habits. Review catches violations. Each layer is necessary, and each layer is fallible. A pipeline shifts some of the enforcement from human judgment to mechanism. The conventions that the pipeline enforces no longer need to be documented in a wiki, because they are enforced at the moment they would be violated. The training for new team members shrinks accordingly, because the environment teaches the conventions through its behavior. Review focuses on the things that pipelines cannot enforce, like architectural choices and code quality, rather than on style and formatting issues that mechanism handles cleanly. The result is a team where attention is concentrated on what is genuinely human work, and the mechanical work is done by the mechanism. This is, in the end, the deeper promise of the engineered Claude Code environment, and pipelines are the mechanism that delivers it.

文档、培训、评审。文档把"规约是什么"写下来;培训把习惯建立起来;评审把违规抓出来。每一层都必要,每一层也都靠不住。流水线把一部分强制工作从"人的判断"挪到了"机制"。被流水线强制住的规约,不再需要写到 wiki 里——因为它会在"差点被违反"的那一刻被强制

住;对新成员的培训也相应瘦身——因为环境会通过自身的行为"教"出规约;评审则得以聚焦在"流水线兜不住的事"上——比如架构选择、代码质量——而不是把力气花在机制本就能干净处理的风格与格式化上。结果是:整个团队的注意力,被集中到那些真正属于"人"的工作;机械的部分由机制去做。这,正是"工程化的 Claude Code 环境"更深一层的承诺;而流水线,是把这份承诺兑现出来的那条机制。

A final observation about pipelines, before this chapter ends, concerns what happens when several pipelines coexist. A user-level pipeline applies to every project. A project-level pipeline applies inside the specific project. The two pipelines combine: every event that fires runs the hooks from both levels, in a defined order. The combined behavior is the union of the two pipelines, which means that the developer can build a stable personal pipeline at the user level and let projects extend it as needed. A team that has agreed on certain project-level conventions can encode them in the project pipeline. Each developer's personal preferences are layered on top through the user pipeline. The combination is a personalized variant of the team standard, and the developer experiences both layers as a single coherent environment. This layered architecture is one of the quieter strengths of the hook system, and developers who design with it in mind end up with environments that are both standardized and personalized, without conflict between the two.

本章收束前,再补一条关于流水线的观察——关于"多条流水线并存"时会发生什么。用户级流水线作用于所有项目;项目级流水线作用于该项目内部。两者是组合的关系:任一事件触发时,两层级的 hook 都会按既定顺序跑一遍。组合行为就是两层流水线之"并",也就是说:开发者可以在用户层搭一条稳定、长期的个人流水线,再让项目按需扩展它。团队就"项目级规约"达成的共识,被编码进项目流水线;每位开发者的个人偏好,通过用户流水线叠在它上面。组合起来,就是"团队标准的个性化变体";而开发者本人体验到的,是两层合为一块"内洽的环境"。这种"分层"架构,是 hook 系统里相当安静的一项强项;按它来设计的开发者,最终会得到一个"既标准化、又个性化"的环境,两层之间互不冲突。

One more observation deserves a place before this chapter closes, about the felt experience of working with a substantial pipeline installed. The pipeline does its work invisibly. The developer does not see the formatting hook fire, because the file simply ends up formatted. The developer does not see the test guard run, because the tests pass and the session continues normally. The developer does not see the audit logger record her

本章收束前,再放一条观察——关于"装上一条相当规模的流水线之后的工作体感"。流水线做的事是"看不见"的:开发者不会"看见"那条格式化 hook 触发,因为文件"自然而然"就格式化好了;不会"看见"那条测试 guard 跑过,因为测试通过,session 照常推进;也不会"看见"审计 logger 把她的

← 前一页

后一页 →

commands, because the log file fills silently in the background. What the developer experiences is a working environment that simply behaves the way she wants it to, as if the conventions she cares about were the natural shape of the tool itself. The configuration is invisible, and the behavior is what remains. After a few weeks of working in such an environment, the developer often forgets that the behavior is the result of configuration at all; the environment feels like the default, and the absence of the configuration on a fresh installation feels strange. This is the deepest sign of a successful pipeline. It has receded into the background, and the work has become foreground. The architecture, once built, asks nothing further of the developer, and the developer's attention is free to focus entirely on what she is making.

命令记下来,因为那份日志是在后台默默填满的。开发者真正"体验到"的,是一个"她想要什么它就怎么表现"的工作环境——仿佛她所看重的那些规约,本就是工具自己的天然形状。配置是不可见的,剩下的只有行为。在这样的环境里工作几周之后,开发者常常会忘记:这种行为其实是配置出来的。环境成了"默认",而一个全新安装、没装这套配置的环境,反倒显得别扭。这就是一条成功流水线最深处的标志:它已经退到背景里,工作被推到了前景。这副架构一旦搭好,不再向开发者索取什么;她的注意力,可以完全聚焦于"自己在造的那个东西"。

A reflection before this chapter and this part of the book end. Hooks are quieter than skills and subagents. They do not produce visible artifacts in most sessions. They sit at the edges of the lifecycle, doing their work without drawing attention, and they earn their place by what they prevent rather than by what they produce. The developer who has installed a thoughtful set of hooks may not notice them for weeks at a time, and that invisibility is exactly what good hook 设计 produces. The hooks that are invisible are the hooks that are working. The day comes, eventually, when a hook fires on a real action that needed to be blocked, or a guard catches a problem that would have shipped, or a notifier surfaces an event that needed attention. On that day, the entire investment in the hook bundle is repaid in a single moment, and the developer recognizes, sometimes for the first time, that the architecture has been protecting her all along.

本章、也是本书这一部分收束前的反思:hook 比 skill、subagent 都更"安静"。在大多数 session 里,它们并不产出可见的工件。它们蹲在生命周期的边缘,默默做事,不引人注目;它们赢得位置,靠的是"它们拦下了什么"而非"它们产出了什么"。装上一套"用心挑选过"的 hook 的开发者,可能连续几周都想不起它们;而这种"隐形",恰恰是好 hook 设计所产生的结果。"看不见"的 hook,才是"在工作"的 hook。终有一天,某条 hook 在一次真该拦的动作上跳出来,或者某条 guard 抓住了一个本该被发出去的问题,又或者某条 notifier 把一件确实需要关注的事顶到台面——就在那一天,装这套 hook 套件的全部投入,以一个瞬间被一次性结清;开发者常常是头一次意识到:这套架构,其实一直在保护她。

Part Four ends here. The reader has, over four chapters, learned the mechanics of hooks, internalized the gate-guard-notifier vocabulary that organizes hook 设计, installed a security-hook bundle, and seen how multiple hooks compose into event pipelines that wrap entire workflows in deterministic guarantees. The third pillar of the framework is now in place. Three pillars stand. The next part of the book takes the architecture as a whole and shows how to build a complete environment in which all three pillars work together. Two end-to-end case studies, each substantial, will demonstrate the architecture in concrete form. By the end of the next part, the reader will have seen the framework not as three separate sets of techniques but as a single coherent system, and she will have the blueprint

第四部分至此收束。读者用四章时间,学会了 hook 的机制、内化了"gate–guard–notifier"这套组织 hook 设计的词汇、装好了一套安全 hook 套件,又亲眼见证了多条 hook 是如何组合成事件流水线、把整条工作流包在确定性保证之中。框架的第三根支柱,至此就位。三根支柱,立住了。本书的下一部分,要把架构作为一个整体来谈:如何在"三根支柱一起工作"的前提下,搭建出一套完整的环境。会有两个端到端的、份量扎实的案例研究,把这套架构以具体形式演给你看。读完下一部分,读者将看到的不再是"三套彼此分立的技术",而是"一个内洽的整体系统";她将拿到一份蓝图,

← 前一页

后一页 →

for building such a system in her own work. That synthesis begins in the chapter ahead.

足以在自己的工作里搭出这样一套系统。这场"综合",从下一章开始。

← 前一页

PART FIVE (Ch 19) →